ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

NGÀNH: CÔNG NGHỆ THÔNG TIN

GVHD: ThS. Phạm Thi Vương

SVTH: Nguyễn Công Quân 3123560070

Nguyễn Quốc Đại 3123560013

Nguyễn Thái Vinh 3123410433

Thành phố Hồ Chí Minh, năm 2026

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

NGÀNH: CÔNG NGHỆ THÔNG TIN

GVHD: ThS. Phạm Thi Vương

SVTH: Nguyễn Công Quân 3123560070

Nguyễn Quốc Đại 3123560013

Nguyễn Thái Vinh 3123410433

Thành phố Hồ Chí Minh, năm 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng đồ án môn học *"Tìm hiểu công nghệ Next.js qua Ứng dụng tra cứu & khám phá phim ảnh"* là công trình nghiên cứu và phát triển độc lập của nhóm sinh viên thực hiện dưới sự hướng dẫn chuyên môn của ThS. Phạm Thi Vương.

Các kết quả kiểm chứng trong báo cáo được ghi theo mã nguồn và lệnh đã chạy. Nhóm không công bố số đo Core Web Vitals, kích thước bundle hoặc Streaming TTFB khi chưa có dữ liệu gốc. Tài liệu và nguồn dữ liệu tham khảo được liệt kê ở cuối báo cáo.

Nhóm xin chịu hoàn toàn trách nhiệm trước Khoa Công nghệ Thông tin và Trường Đại học Sài Gòn về tính trung thực và liêm chính học thuật của đồ án này.

Thành phố Hồ Chí Minh, ngày 03 tháng 10 năm 2026

Đại diện nhóm sinh viên thực hiện

Nguyễn Công Quân **Nguyễn Quốc Đại** **Nguyễn Thái Vinh**

LỜI CẢM ƠN

Để hoàn thành đồ án môn học này một cách chỉn chu và đạt được chiều sâu kỹ thuật cao, nhóm chúng em đã nhận được sự quan tâm, tạo điều kiện và hướng dẫn quý báu từ quý Thầy, Cô và các tổ chức liên quan.

Trước hết, nhóm xin gửi lời tri ân chân thành và sâu sắc nhất đến **ThS. Phạm Thi Vương** – Giảng viên phụ trách học phần Các Công nghệ Lập trình Hiện đại. Thầy không chỉ truyền đạt những kiến thức nền tảng vững chắc và định hướng tiếp cận công nghệ đúng đắn: *"Trọng tâm của môn học là làm chủ CÔNG NGHỆ, không phải sa đà vào nghiệp vụ phức tạp; ứng dụng xây dựng chỉ là phương tiện để áp dụng và minh họa"*, mà còn luôn nhiệt tình theo sát, giải đáp các vướng mắc kỹ thuật hóc búa và định hướng nhóm giải quyết bài toán cốt lõi.

Nhóm cũng xin chân thành cảm ơn Ban Chủ nhiệm Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn đã xây dựng chương trình đào tạo hiện đại, tạo lập môi trường học tập thực tế giúp sinh viên có cơ hội tiếp cận và làm chủ những công nghệ mới nhất trên thế giới.

Cuối cùng, nhóm xin gửi lời cảm ơn đến cộng đồng mã nguồn mở Next.js (Vercel), React Core Team và nền tảng The Movie Database (TMDB) đã cung cấp các tài liệu kỹ thuật chuẩn xác và nguồn dữ liệu mở chất lượng cao để nhóm hoàn thiện đồ án này.

Dù đã rất nỗ lực nghiên cứu và hoàn thiện, song do thời gian có hạn và kiến thức thực tế còn đang trong quá trình tích lũy, đồ án chắc chắn không tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những lời nhận xét, góp ý quý báu của Thầy Cô để sản phẩm ngày càng hoàn thiện hơn.

Nhóm sinh viên thực hiện

TÓM TẮT ĐỒ ÁN

Báo cáo nghiên cứu Next.js 16 (App Router) qua ứng dụng MovieHub sử dụng React 19, TypeScript và dữ liệu phim từ TMDB API. Trọng tâm là mô hình kết xuất của Next.js, ranh giới Server/Client Component, dữ liệu phía server, định tuyến, cache và Streaming với React Suspense; nghiệp vụ tra cứu phim chỉ là bối cảnh minh họa.

Trang chủ là Server Component, lấy năm nhóm dữ liệu song song bằng `Promise.all`. Các thẻ `MovieCard` được tạo ở server và truyền dưới dạng phần tử React cho `MovieRow` phía client để thanh cuộn xử lý sự kiện mà không nhập trực tiếp mã của thẻ phim vào client bundle. Trang `/movie/[id]` dùng tham số bất đồng bộ, metadata động và hai ranh giới Suspense cho diễn viên và phim tương tự. Trang `/browse` và `/search` lưu bộ lọc trên URL. Watchlist dùng Server Action ghi cookie `HttpOnly`; tìm kiếm gợi ý dùng Route Handler GET cùng debounce và `AbortController`.

Bằng chứng hiện có gồm mã nguồn, kiểm thử tự động, kết quả lint/type-check và bảng phân loại route của bản build production. Báo cáo không khẳng định điểm Lighthouse, kích thước bundle hay mức cải thiện TTFB khi chưa có dữ liệu đo gốc và kịch bản đối chứng có thể tái lập. Các giới hạn này được nêu rõ tại Chương 4.

Từ khóa: *Next.js 16, App Router, React Server Components, Streaming, Suspense, ISR, Server Actions, Route Handler, TMDB.*

MỤC LỤC

LỜI CAM ĐOAN i

LỜI CẢM ƠN ii

TÓM TẮT ĐỒ ÁN iii

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ vii

DANH MỤC BẢNG BIỂU ix

DANH MỤC HÌNH ẢNH x

LỜI MỞ ĐẦU 1

1 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG 4

1.1. Tổng quan về Next.js và lịch sử phát triển 4

1.1.1. Bối cảnh ra đời và hạn chế của React Single Page Application (SPA) 4

1.1.2. Bước chuyển đổi với Pages Router (Next.js 1 – 12) 4

1.1.3. Kỷ nguyên mới với App Router và Server-Driven UI (Next.js 13 – 16) 5

1.1.4. So sánh Next.js App Router với các giải pháp hiện đại khác 6

1.1.5. Phản biện kỹ thuật: Khi nào KHÔNG nên sử dụng Next.js? 6

1.2. Kiến trúc React Server Components (RSC) và Client Components 7

1.2.1. Bản chất và nguyên lý hoạt động của RSC 7

1.2.2. Định dạng dữ liệu RSC Payload và cây render hỗn hợp 7

1.2.3. Chỉ thị `"use client"` và quy tắc Composition Pattern 8

1.3. Cơ chế Caching và Data Fetching đa tầng trong Next.js 16 9

1.3.1. Tính hợp lệ của API `fetch(url, { next: { revalidate } })` 9

1.3.2. Cơ chế Cache Components (`"use cache"` và `cacheLife`) trong Next.js 16 9

1.3.3. Tác động của việc đọc `cookies()` đến quá trình Render và Caching 10

1.3.4. Mối quan hệ tương hỗ giữa Streaming Suspense và Data Caching . 10

1.3.5. Các cơ chế nền tảng bổ trợ: Hydration, Vòng đời Hook và URL State 10

1.4. Kỹ thuật Streaming UI và React Suspense 11

1.4.1. Hạn chế của SSR truyền thống (All-or-Nothing Problem) 11

1.4.2. Cơ chế Streaming HTML lũy tiến 11

1.5. Cơ chế Server Actions và xử lý Mutation dữ liệu 12

1.5.1. Kiến trúc Remote Procedure Call (RPC) 12

1.5.2. Bảo mật dữ liệu, kiểm thực đầu vào và giới hạn Cookie HttpOnly . 12
1.6. Hệ sinh thái công nghệ bổ trợ 13
1.6.1. Thư viện React 19 13
1.6.2. Framework giao diện Tailwind CSS v4 13
1.6.3. Ngôn ngữ lập trình TypeScript 13
1.7. Nền tảng dữ liệu mở The Movie Database (TMDB) API 14

2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 15
2.1. Khảo sát và đặc tả yêu cầu hệ thống 15
2.1.1. Yêu cầu chức năng (Functional Requirements) 15
2.1.2. Yêu cầu phi chức năng (Non-Functional Requirements) 16
2.2. Kiến trúc hệ thống tổng thể 16
2.2.1. Mô hình kiến trúc 3 tầng phân lớp 16
2.2.2. Cấu trúc thư mục dự án theo chuẩn App Router 17
2.3. Thiết kế tích hợp dịch vụ và luồng dữ liệu 18
2.3.1. Kiến trúc tầng dịch vụ `services/client.ts` 18
2.3.2. Giải pháp kiến trúc DNS Fallback Resolver 18
2.3.3. Phân tầng chiến lược Caching và Revalidation 19
2.4. Thiết kế giao diện và trải nghiệm người dùng (UI/UX) 19
2.4.1. Hệ thống thiết kế chuẩn điện ảnh (Netflix Dark Cinema Design System) 19
2.4.2. Sơ đồ điều hướng và luồng trải nghiệm người dùng 20
2.5. Môi trường và công cụ phát triển 21

3 HIỆN THỰC HỆ THỐNG 22
3.1. Cài đặt và cấu hình dự án 22
3.2. Hiện thực Tầng 1A: Các tính năng cốt lõi bắt buộc 22
3.2.1. Phân định ranh giới Server/Client Component 22
3.2.2. Server-side Data Fetching và song song hóa với `Promise.all` . . . 23
3.2.3. Dynamic Routing (`/movie/[id]`) và Dynamic SEO Metadata . . . 23
3.3. Hiện thực Tầng 1B: Các kỹ thuật chuyên sâu và nâng cao 24
3.3.1. Server Action cho mutation và Route Handler cho tìm kiếm 24
3.3.2. Streaming Suspense và Skeleton Loading States 24
3.3.3. Tối ưu hóa hình ảnh với `next/image` và YouTube Player API 25
3.4. Hiện thực các module giao diện người dùng chính 25
3.4.1. Module Hero Banner & Trailer Modal xem trailer tức thì 25
3.4.2. Module MovieRow Slider & Carousel cuộn vô tận 25
3.4.3. Module FilterBar & Trang Khám phá `/browse` 25

3.4.4. Module Thanh điều hướng nâng cao (`NavbarDropdown`, `NavSearchBar`) 26
3.4.5. Module Trang Phim bộ (`/tv`) và Khám phá diễn viên 26
3.5. Hiện thực cơ chế tự phục hồi và khử trùng lặp dữ liệu 26
3.5.1. Giới hạn của cơ chế DNS dự phòng 26
3.5.2. Thuật toán khử trùng lặp phim (`deduplicateMovies`) 27
3.6. Tổ chức phát triển và tiến độ thực hiện 27
3.6.1. Bảng phân công trách nhiệm nhóm 27
3.6.2. Tiến độ thực hiện theo tuần (Tóm tắt kế hoạch 7 tuần) 28
3.6.3. Quy trình làm việc nhóm với Git Flow 29

4 KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ 31
4.1. Phương pháp và giới hạn của bằng chứng 31
4.2. Kiểm tra tự động và khả năng chạy lại 31
4.3. Bằng chứng về mô hình render và cache 32
4.4. Ma trận đáp ứng yêu cầu môn học 33

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 35

TÀI LIỆU THAM KHẢO 36

PHỤ LỤC 37

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Thuật ngữ	Định nghĩa và Giải thích chuyên môn
API	*Application Programming Interface* – Giao diện lập trình ứng dụng cho phép các hệ thống giao tiếp với nhau.
CLS	*Cumulative Layout Shift* – Chỉ số đo lường mức độ ổn định bố cục thị giác của trang web (tránh hiện tượng gián đoạn, giật màn hình khi tải ảnh).
CSR	*Client-Side Rendering* – Mô hình kết xuất phía máy khách truyền thống trong React SPA, trình duyệt nhận tệp HTML rỗng và tự nạp JavaScript để sinh cây giao diện DOM.
DNS	*Domain Name System* – Hệ thống phân giải tên miền mạng thành địa chỉ IP máy chủ.
DOM	*Document Object Model* – Mô hình đối tượng tài liệu dạng cây biểu diễn cấu trúc trang web của trình duyệt.
FCP	*First Contentful Paint* – Thời điểm trình duyệt bắt đầu vẽ phần tử nội dung DOM đầu tiên lên màn hình.
ISR	*Incremental Static Regeneration* – Cơ chế tái tạo trang tĩnh định kỳ theo thời gian chỉ định (revalidate) mà không cần biên dịch lại toàn bộ dự án.
LCP	*Largest Contentful Paint* – Chỉ số đo thời gian kết xuất phần tử nội dung trực quan lớn nhất trong khung nhìn người dùng.
PR	*Pull Request* – Yêu cầu đóng góp và hợp nhất nhánh tính năng vào nhánh chính có kiểm duyệt mã nguồn chéo.
REST	*Representational State Transfer* – Kiểu kiến trúc tiêu chuẩn cho các dịch vụ web giao tiếp qua giao thức HTTP.
RPC	*Remote Procedure Call* – Giao thức cho phép gọi trực tiếp một hàm thực thi từ xa trên máy chủ qua mạng, nền tảng của Server Actions.
RSC	*React Server Components* – Component chạy trên máy chủ; mã chỉ dùng ở server không phải tải như JavaScript của Client Component.
SEO	*Search Engine Optimization* – Quá trình tối ưu hóa mã nguồn và nội dung để tăng thứ hạng hiển thị trên công cụ tìm kiếm.
SPA	*Single Page Application* – Ứng dụng web đơn trang kết xuất và định tuyến hoàn toàn tại trình duyệt client.
SSG	*Static Site Generation* – Mô hình tạo sẵn toàn bộ tài liệu HTML tĩnh ngay từ thời điểm biên dịch ứng dụng (Build-time rendering).

Thuật ngữ	Định nghĩa và Giải thích chuyên môn
SSR	*Server-Side Rendering* – Mô hình kết xuất HTML động từ máy chủ cho mỗi lượt truy cập (Request-time rendering).
TMDB	*The Movie Database* – Nguồn cung cấp RESTful API công cộng dữ liệu phim điện ảnh và truyền hình trực tuyến.
TTFB	*Time to First Byte* – Thời gian phản hồi từ lúc client gửi request đến khi nhận được byte dữ liệu đầu tiên từ server.

DANH SÁCH BẢNG

1.1 So sánh đặc trưng kỹ thuật giữa CSR, Pages Router và App Router (Nguồn: Tổng hợp từ [1], [2], [7]) 5
1.2 Đối sánh kỹ thuật giữa Next.js App Router và các giải pháp web hiện đại (Nguồn: Nhóm tác giả tổng hợp) 6
2.1 Đặc tả các yêu cầu chức năng chính của hệ thống MovieHub (Nguồn: Nhóm tác giả đề xuất) 15
2.2 Thời hạn revalidate của các yêu cầu TMDB trong tầng services 19
2.3 Bảng mã màu và thông số thiết kế Netflix Dark Cinema (Nguồn: Trích xuất từ tài liệu DESIGN.md của dự án) 20
3.1 Bảng phân công trách nhiệm và tỷ lệ đóng góp của các thành viên (Nguồn: Nhóm tác giả thống nhất) 28
3.2 Các nhóm công việc trong kế hoạch 7 tuần và hiện vật tương ứng 29
4.1 Các kiểm tra có thể thực thi lại trên mã nguồn hiện tại 31
4.2 Phân loại route từ `npm run build` trên bản mã nguồn hiện tại 32
4.3 Đối chiếu yêu cầu và tệp minh chứng trong repo 33

DANH SÁCH HÌNH VẼ

1.1 Sơ đồ luồng kết xuất React Server Components và đóng gói RSC Payload (Nguồn: Xây dựng dựa trên đặc tả kỹ thuật RSC [7]) 8
2.1 Sơ đồ kiến trúc 3 tầng phân lớp hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế) 17
2.2 Sơ đồ luồng điều hướng trải nghiệm người dùng trong hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế) 20

LỜI MỞ ĐẦU

1. Tính cấp thiết của đề tài. Ứng dụng một trang (SPA) kết xuất chủ yếu trên trình duyệt tạo trải nghiệm tương tác linh hoạt, nhưng cần quản lý chi phí JavaScript và cách cung cấp nội dung ban đầu. Các vấn đề cần xem xét gồm:

- **Chi phí JavaScript:** Trình duyệt phải tải, phân tích cú pháp và thực thi mã dành cho các thành phần tương tác; chi phí này phụ thuộc vào từng ứng dụng.
- **Thời gian hiển thị:** Nếu nội dung chỉ được tạo sau khi JavaScript chạy, FCP và LCP có thể bị ảnh hưởng; cần đo trên cùng thiết bị và điều kiện mạng để kết luận.
- **Khả năng đọc nội dung và metadata:** HTML có sẵn từ máy chủ giúp trình thu thập và công cụ xem trước liên kết tiếp cận nội dung ban đầu; mức độ hỗ trợ JavaScript của từng công cụ không giống nhau.
- **Quản lý khóa dịch vụ:** Nếu trình duyệt gọi API bên thứ ba bằng khóa bí mật, khóa đó sẽ hiện diện trong yêu cầu mạng; các yêu cầu cần khóa nên thực hiện trên máy chủ.

Next.js [1] với App Router và React Server Components (RSC) [7] cho phép tạo nội dung trên máy chủ và truyền kết quả xuống trình duyệt. Mã của Server Component không cần đưa vào JavaScript bundle của client, nhưng ứng dụng vẫn cần JavaScript cho Client Components và cơ chế điều hướng. Việc giữ khóa TMDB trong tầng dịch vụ phía server giảm nguy cơ lộ khóa qua mã và yêu cầu mạng của trình duyệt.

Quán triệt tư tưởng chỉ đạo của học phần Các Công nghệ Lập trình Hiện đại: *"Trọng tâm của môn học là làm chủ CÔNG NGHỆ, không phải sa đà vào nghiệp vụ phức tạp; ứng dụng xây dựng chỉ là phương tiện để áp dụng và minh họa"*, nhóm sinh viên chúng em thực hiện đề tài: **"Tìm hiểu công nghệ Next.js qua Ứng dụng tra cứu & khám phá phim ảnh"**. Bằng việc lựa chọn nguồn dữ liệu có sẵn từ The Movie Database (TMDB) API v3 [3], toàn bộ nguồn lực và công sức của nhóm được tập trung tuyệt đối vào việc làm chủ các cơ chế kỹ thuật cốt lõi của Next.js.

2. Mục tiêu nghiên cứu của đồ án. Đồ án bám sát khung chuẩn đánh giá 3 tầng của môn học:

1. **Tầng 1A (Mục tiêu bắt buộc lõi):**
    - Nắm vững cấu trúc thư mục App Router định tuyến theo tệp tin (`app/layout.tsx`, `app/page.tsx`).
    - Phân định rõ ràng ranh giới giữa Server Components (chạy mặc định trên máy chủ) và Client Components (khai báo chỉ thị `"use client"`).
    - Gọi TMDB tại tầng dịch vụ phía máy chủ và không truyền khóa API xuống Client Components.

2. **Tầng 1B (Mục tiêu nâng cao theo đặc thù ứng dụng):**
    - Làm chủ định tuyến động (Dynamic Routing) `/movie/[id]` xử lý tham số bất đồng bộ dạng Promise (bắt đầu từ Next.js 15 và trở thành quy chuẩn bắt buộc trong Next.js 16).
    - Triển khai Streaming UI với React Suspense và fallback cho các phần dữ liệu phụ; đánh giá thời điểm hiển thị bằng trace khi có dữ liệu đo.
    - Ứng dụng cơ chế Caching dữ liệu tĩnh có kiểm soát thời hạn Incremental Static Regeneration (ISR) trong tầng dịch vụ.
    - Xây dựng các chức năng tương tác dữ liệu bảo mật thông qua Server Actions (`"use server"`) đọc/ghi Cookie an toàn.
    - Dùng `next/image` và khung ảnh có tỉ lệ xác định để hạn chế dịch chuyển bố cục.
    - Tự động hóa SEO thông qua hàm `generateMetadata` trích xuất thông tin động.
3. **Tầng 2 (Mục tiêu chuẩn kỹ nghệ phần mềm):**
    - Quản lý mã nguồn theo mô hình Git Flow phân nhánh chuyên nghiệp, kiểm duyệt mã thông qua Pull Request.
    - Đảm bảo tính nhất quán và an toàn mã nguồn với TypeScript chế độ nghiêm ngặt (Strict Mode).
    - Xử lý lỗi mạng và đánh giá giới hạn của mã phân giải DNS dự phòng trong Node.js.
    - Tuân thủ nghiêm ngặt hệ thống thiết kế chuẩn điện ảnh Netflix Dark Cinema (`DESIGN.md`).

3. Đối tượng và phạm vi nghiên cứu.

- **Đối tượng nghiên cứu:** Nền tảng Next.js phiên bản 16.3.6 (App Router), thư viện React 19.2.8, ngôn ngữ TypeScript, framework CSS tiện ích Tailwind CSS v4, và giao diện lập trình ứng dụng The Movie Database (TMDB) REST API v3.
- **Phạm vi ứng dụng:** Xây dựng ứng dụng web **MovieHub** phục vụ việc tra cứu, tìm kiếm, lọc đa tiêu chí, xem chi tiết và quản lý danh sách theo dõi phim ảnh. Ứng dụng tập trung khai thác và trình diễn các kỹ thuật Next.js; không đi sâu vào việc phát triển hạ tầng backend cơ sở dữ liệu riêng mà tận dụng nguồn dữ liệu phong phú từ TMDB.
- **Thời gian thực hiện:** Dự án được tiến hành trong thời gian 7 tuần theo kế hoạch đào tạo của học phần.

4. Phương pháp nghiên cứu.

- **Phương pháp nghiên cứu lý thuyết:** Nghiên cứu tài liệu chính thống (Official Documentation) từ Vercel, React Core Team và W3C; phân tích các thông số kỹ thuật (RFC)

của React Server Components và cơ chế HTTP Streaming.

- **Phương pháp thực nghiệm phát triển:** Áp dụng quy trình kỹ nghệ phần mềm chuyên nghiệp, xây dựng mã nguồn từ cơ bản đến nâng cao theo từng sprint tuần, kiểm thử đối chứng trên môi trường thực tế.
- **Phương pháp kiểm chứng:** Đối chiếu mã với yêu cầu; chạy lint, type-check, test và production build. Các phép đo Lighthouse, bundle và TTFB được mô tả như công việc tiếp theo vì chưa có tệp kết quả gốc.

5. Bố cục của báo cáo đồ án. Nội dung báo cáo được tổ chức thành 4 chương chính như sau:

- **Chương 1: Cơ sở lý thuyết và công nghệ sử dụng** – Trình bày tổng quan về Next.js, kiến trúc App Router, React Server Components, cơ chế Caching, Streaming, Server Actions và các công nghệ phụ trợ.
- **Chương 2: Phân tích và thiết kế hệ thống** – Đặc tả các yêu cầu chức năng và phi chức năng, thiết kế kiến trúc phân lớp, cấu trúc thư mục, luồng dữ liệu, giải pháp DNS Fallback và hệ thống thiết kế UI/UX.
- **Chương 3: Hiện thực hệ thống** – Chi tiết quá trình cài đặt, hiện thực các tính năng Tầng 1A, Tầng 1B, các module giao diện nổi bật, cơ chế chịu lỗi mạng, cùng quy trình phân công và tiến độ thực hiện.
- **Chương 4: Kiểm thử và đánh giá kết quả** – Trình bày bằng chứng kiểm chứng hiện có, giới hạn của phép kiểm thử và ma trận đối sánh yêu cầu môn học.
- **Kết luận và hướng phát triển** – Tổng kết các kết quả đạt được, chỉ ra những hạn chế và đề xuất hướng mở rộng trong tương lai.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1.1. Tổng quan về Next.js và lịch sử phát triển

1.1.1. Bối cảnh ra đời và hạn chế của React Single Page Application (SPA)

Kể từ khi được Facebook giới thiệu vào năm 2013, thư viện React đã tạo nên cuộc cách mạng trong thiết kế giao diện nhờ mô hình Virtual DOM và tư duy phát triển dựa trên Component. Tuy nhiên, mô hình ứng dụng đơn trang (SPA) dựa trên cơ chế Client-Side Rendering (CSR) đã bộc lộ những nhược điểm lớn khi mở rộng quy mô:

1. **Hiện tượng trang trắng ban đầu (Blank Page):** Trình duyệt nhận về tệp tin HTML gần như rỗng chỉ chứa một phần tử `<div id="root"></div>`. Người dùng phải đợi trình duyệt tải xong toàn bộ gói mã nguồn JavaScript, sau đó chạy mã để dựng cây DOM thực tế.
2. **Tải trọng mạng và chi phí xử lý của thiết bị di động:** Các gói thư viện phụ thuộc (npm packages) đều bị đóng gói dồn vào bundle của client. Trên các thiết bị di động cấu hình yếu, việc phân tích cú pháp (parse) và biên dịch (compile) tệp JavaScript nặng hàng megabyte gây đơ giật giao diện và kéo dài chỉ số Time to Interactive (TTI).
3. **Thách thức về SEO và Social Sharing:** Các hệ thống lập chỉ mục tìm kiếm và robot thu thập dữ liệu mạng xã hội (Open Graph) thường không thực thi hoàn chỉnh các luồng JavaScript bất đồng bộ, khiến website bị xếp hạng thấp hoặc không hiển thị được ảnh đại diện xem trước.

1.1.2. Bước chuyển đổi với Pages Router (Next.js 1 – 12)

Nhằm khắc phục các điểm yếu của React SPA, Vercel đã giới thiệu nền tảng Next.js vào năm 2016 [1]. Kiến trúc Pages Router tổ chức các trang web tương ứng với các tệp tin trong thư mục `pages/`, đồng thời cung cấp hai cơ chế tiền kết xuất mạnh mẽ:

- **Server-Side Rendering (SSR) với `getServerSideProps`:** Máy chủ thực thi lấy dữ liệu và kết xuất mã HTML hoàn chỉnh cho từng request của người dùng trước khi gửi về client.
- **Static Site Generation (SSG) với `getStaticProps`:** Máy chủ tạo sẵn tài liệu HTML ngay tại thời điểm biên dịch ứng dụng (Build-time), mang lại tốc độ phản hồi cực nhanh thông qua mạng phân phối nội dung (CDN).

Tuy nhiên, Pages Router vẫn tồn tại hạn chế về mặt kiến trúc gọi là **Chi phí Hydration toàn bộ (Hydration Overhead)**. Sau khi máy chủ gửi tài liệu HTML về client, trình duyệt vẫn bắt buộc phải tải về toàn bộ mã nguồn JavaScript của toàn bộ trang để tiến hành quá

trình "Hydrate" – gắn các trình lắng nghe sự kiện (Event Listeners) vào các nút DOM tĩnh. Điều này đồng nghĩa với việc mã nguồn của các component chỉ hiển thị nội dung tĩnh (như Header, Footer, Sidebar) vẫn bị gửi xuống client một cách dư thừa.

1.1.3. Kỷ nguyên mới với App Router và Server-Driven UI (Next.js 13 – 16)

Nhận thức được giới hạn của kiến trúc cũ, đội ngũ kỹ sư Vercel phối hợp chặt chẽ cùng React Core Team đã phát triển kiến trúc **App Router**, được chính thức đưa vào sản xuất từ Next.js 13 và hoàn thiện vững chắc trong Next.js 16 [1].

App Router được xây dựng xung quanh khái niệm **React Server Components (RSC)** [7]. Trọng tâm kết xuất được chuyển dịch triệt để về phía máy chủ (Server-Driven UI). Trong mô hình này, các component mặc định chỉ chạy trên máy chủ, cho phép truy cập trực tiếp vào hệ thống tệp, cơ sở dữ liệu và biến môi trường bảo mật mà không gửi một dòng mã lệnh JavaScript nào của component đó xuống trình duyệt.

Bảng 1.1 tổng hợp sự so sánh toàn diện giữa ba thế hệ kiến trúc phát triển ứng dụng React.

Bảng 1.1. So sánh đặc trưng kỹ thuật giữa CSR, Pages Router và App Router (Nguồn: Tổng hợp từ [1], [2], [7])

Tiêu chí so sánh	**React CSR (SPA)**	**Next.js Pages Router**	**Next.js App Router**
Vị trí kết xuất chính	Trình duyệt Client	Server & Client Hydrate	Server (mặc định) + Client
Tải trọng JS Client	Rất lớn (toàn bộ app)	Lớn (toàn trang nạp)	Tối ưu (Zero-bundle RSC)
Khả năng SEO	Rất kém (HTML rỗng)	Tốt (SSR/SSG tĩnh)	Tối ưu xuất sắc
Cơ chế Caching	Trình duyệt thủ công	HTTP Cache đơn giản	Caching 4 tầng tự động
Hỗ trợ Streaming	Không hỗ trợ	Không có	React Suspense Streaming
Xử lý Mutation	Gọi REST API Route	Tạo thư mục `pages/api`	Server Actions trực tiếp
Bảo mật API Key	Dễ lộ trong mã client	Phải qua API trung gian	Giữ hoàn toàn trên Server

1.1.4. So sánh Next.js App Router với các giải pháp hiện đại khác

Để có cái nhìn khách quan và toàn diện về bức tranh công nghệ web hiện đại, Bảng 1.2 đối sánh Next.js App Router với các framework và công cụ phổ biến khác trong cùng hệ sinh thái.

Bảng 1.2. Đối sánh kỹ thuật giữa Next.js App Router và các giải pháp web hiện đại (Nguồn: Nhóm tác giả tổng hợp)

Tiêu chí	Next.js 16 (App Router)	React + Vite (CSR)	Nuxt 4 (Vue SSR)	Astro (Islands)
Mô hình render	Hybrid (RSC + SSR + Client)	Pure Client-Side Rendering	Universal SSR / Nitro	Component Islands (Zero JS default)
Tải trọng JS	Tối ưu (Zero-bundle RSC)	Rất lớn theo quy mô app	Trung bình (Full Hydrate)	Cực nhỏ (chỉ hydrate island tương tác)
Độ phức tạp	Cao (phân định Server/Client)	Thấp, dễ tiếp cận	Trung bình	Thấp đến trung bình
Mục tiêu tối ưu	Ứng dụng lớn, SEO, Dynamic Data	Single Page App, Internal tools	Đa dụng hệ Vue	Nội dung tĩnh, tin tức, thương mại nhẹ
Ràng buộc hạ tầng	Khuyến nghị Node.js/Vercel	Bất kỳ Static Hosting (S3/CF)	Linh hoạt (Nitro Engine)	Static hoặc Adapter nhẹ

1.1.5. Phản biện kỹ thuật: Khi nào KHÔNG nên sử dụng Next.js?

Dựa trên trải nghiệm thực tế trong quá trình phát triển dự án MovieHub, nhóm nhận thấy Next.js không phải là giải pháp vạn năng cho mọi bài toán công nghệ. Dưới đây là các kịch bản thực tế mà việc sử dụng Next.js mang lại nhiều bất lợi hơn là ưu điểm:

1. **Ứng dụng nội bộ hoặc Bảng điều khiển quản trị (Internal Dashboard / CMS sau đăng nhập):** Các hệ thống này hoàn toàn không có nhu cầu tối ưu hóa công cụ tìm kiếm (SEO) hay chia sẻ liên kết mạng xã hội. Việc sử dụng Next.js làm gia tăng độ phức tạp hạ tầng máy chủ và rủi ro Hydration Mismatch một cách không cần thiết; một giải pháp Single Page Application thuần với React + Vite sẽ nhẹ nhàng, dễ bảo trì và tiết kiệm chi phí vận hành hơn rất nhiều.
2. **Website nội dung tĩnh đơn giản (Blog cá nhân, Landing Page, Tài liệu hướng dẫn):** Nếu không cần tương tác phía client, một static site generator như Astro có thể

tạo trang tĩnh với ít JavaScript hơn; không cần vận hành máy chủ Node.js chỉ để phục vụ nội dung hầu như không biến động.

3. **Đội ngũ phát triển chưa quen với ranh giới Server/Client và chi phí hạ tầng:** Sự phân chia nghiêm ngặt giữa môi trường Node.js và trình duyệt (Server Components không có window/DOM/useEffect, async params, Server Actions POST queue) tạo ra một rào cản học tập (Learning Curve) dốc đứng. Ngoài ra, việc tự triển khai Next.js (Self-hosting ngoài Vercel) đòi hỏi cấu hình phức tạp về Cache Handler, ISR phân tán và duy trì tiến trình Node.js liên tục, dẫn đến chi phí vận hành cao hơn so với lưu trữ tĩnh trên CDN.

1.2. Kiến trúc React Server Components (RSC) và Client Components

1.2.1. Bản chất và nguyên lý hoạt động của RSC

Khác với component React truyền thống (chạy trên cả server và client trong quá trình SSR), **React Server Component (RSC)** là một loại component đặc biệt chỉ thực thi duy nhất trên môi trường máy chủ [7].

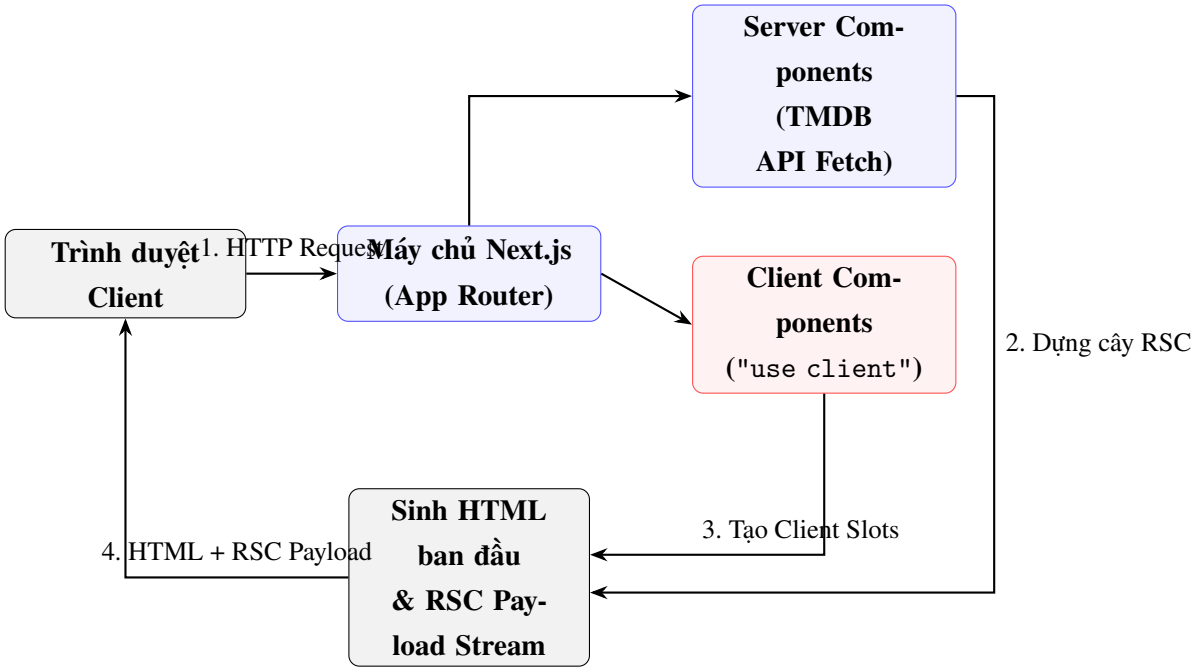
Khi một yêu cầu được gửi tới máy chủ Next.js:

1. Máy chủ thực thi hàm component (hỗ trợ hàm bất đồng bộ `async/await` trực tiếp ở cấp cao nhất).
2. Máy chủ gọi dữ liệu từ cơ sở dữ liệu hoặc dịch vụ bên ngoài (TMDB API) mà không cần dùng hook `useEffect`.
3. Thay vì sinh ra mã JavaScript để gửi về trình duyệt, máy chủ tuần tự hóa (serialize) cây component thành một luồng dữ liệu nhị phân/văn bản có cấu trúc gọi là **RSC Payload**.
4. RSC Payload chứa cấu trúc của cây giao diện, các thuộc tính props được truyền và vị trí đánh dấu (placeholders) nơi các Client Component cần được nạp và hydrate.

Các module chỉ được Server Components nhập sẽ không tham gia vào module graph của Client Components. Tuy nhiên, thư viện được một Client Component nhập trực tiếp vẫn có thể vào client bundle; kích thước thực tế cần kiểm tra bằng analyzer.

1.2.2. Định dạng dữ liệu RSC Payload và cây render hỗn hợp

Hình 1.1 mô tả chi tiết luồng xử lý và sự tương tác giữa Server Components, Client Components và định dạng dữ liệu RSC Payload.



Hình 1.1. Sơ đồ luồng kết xuất React Server Components và đóng gói RSC Payload (Nguồn: Xây dựng dựa trên đặc tả kỹ thuật RSC [7])

1.2.3. Chỉ thị `"use client"` và quy tắc Composition Pattern

Trong Next.js App Router, mọi tệp tin component mặc định là Server Component. Để chuyển đổi một component sang Client Component, lập trình viên phải đặt chỉ thị `"use client"` ở dòng đầu tiên của tệp tin. Chỉ thị này xác định một **Ranh giới mạng (Network Boundary)**, báo hiệu cho trình biên dịch rằng toàn bộ component này cùng các module được nó import sẽ được đóng gói gửi về client để thực hiện hydration.

Các trường hợp bắt buộc phải sử dụng Client Component bao gồm:

- Cần lắng nghe sự kiện tương tác của người dùng: `onClick`, `onChange`, `onSubmit`.
- Cần quản lý trạng thái động hoặc vòng đời: `useState`, `useReducer`, `useEffect`.
- Cần truy cập các API riêng của trình duyệt: `localStorage`, `sessionStorage`, `window`, `document`.
- Cần sử dụng các React Custom Hooks quản lý giao diện tương tác.

Để duy trì hiệu năng tối ưu, đồ án tuân thủ nghiêm ngặt mô hình **Composition Pattern**: Đẩy các Client Component xuống các nút lá nhỏ nhất của cây giao diện (ví dụ: nút bấm mở modal `TrailerModal`, thanh cuộn `MovieRow`), giữ cho các component cha (như `app/page.tsx`, `app/layout.tsx`) là Server Components để thực hiện lấy dữ liệu tập trung.

1.3. Cơ chế Caching và Data Fetching đa tầng trong Next.js 16

Dự án MovieHub được xây dựng trên nền tảng **Next.js 16.3.6** và **React 19.2.8**. Một trong những thay đổi quan trọng nhất của Next.js kể từ phiên bản 15 và 16 là sự chuyển dịch mô hình bộ nhớ đệm: Hàm `fetch` mặc định không còn tự động lưu đệm vĩnh viễn (`force-cache`) như thời Next.js 14, mà chuyển sang mặc định `no-store` trừ khi lập trình viên chủ động khai báo chiến lược lưu trữ.

1.3.1. Tính hợp lệ của API `fetch(url, { next: { revalidate } })`

Một số quan điểm cho rằng cú pháp `fetch(url, { next: { revalidate: seconds } })` là cơ chế cũ của Next.js 13/14 và bị thay thế hoàn toàn trong Next.js 16. Tuy nhiên, theo tài liệu đặc tả chính thức của Vercel (https://nextjs.org/docs/app/api-reference/functions/fetch), đây vẫn là **API chuẩn mực, ổn định (Stable API) và hoàn toàn hợp lệ** trong Next.js 16 để điều khiển **Data Cache ở cấp độ yêu cầu mạng (Request-level Caching)**:

$$\text{fetch}(\text{url}, \{\text{next} : \{\text{revalidate} : 3600\}\}) \tag{1.1}$$

Nguyên lý hoạt động theo cơ chế **Stale-While-Revalidate**:

- Trong khoảng thời gian revalidate (ví dụ 3600 giây), yêu cầu có cùng khóa cache có thể dùng lại dữ liệu đã lưu; báo cáo chưa đo độ trễ cache hit.
- Sau khi hết hạn TTL, khi có lượt truy cập mới, máy chủ phục vụ bản dữ liệu cũ trước (stale), đồng thời kích hoạt một tiến trình nền bất đồng bộ (Background Revalidation) để gọi API TMDB lấy dữ liệu mới và cập nhật bản ghi cache.
- Lượt truy cập tiếp theo sẽ nhận được dữ liệu hoàn toàn mới mà không gây nghẽn luồng xử lý chính.

1.3.2. Cơ chế Cache Components (`"use cache"` và `cacheLife`) trong Next.js 16

Bên cạnh Request Data Cache truyền thống, Next.js 16 giới thiệu kiến trúc bộ nhớ đệm cấp độ component mang tên **Cache Components** (trước đây là cờ thử nghiệm `dynamicIO`):

- **Trạng thái trong dự án:** `next.config.ts` không bật `cacheComponents`; mã hiện tại dùng `fetch` với `next.revalidate`, không dùng `"use cache"` hay `cacheLife`.
- **Cơ chế hoạt động khi được kích hoạt:** Nếu bật `cacheComponents: true`, lập trình viên sử dụng chỉ thị `"use cache"` ở đầu async function hoặc component, kết hợp hàm cấu hình vòng đời `cacheLife()`:
  - `cacheLife("seconds")`: `revalidate: 1s, expire: 60s`.
  - `cacheLife("minutes")`: `revalidate: 60s, expire: 1h`.

- `cacheLife("hours")`: `revalidate: 1h, expire: 24h`.
- `cacheLife("days")`: `revalidate: 24h, expire: 7d`.
- `cacheLife("default")`: dùng profile mặc định do cấu hình Next.js xác định.

Ưu điểm của Cache Components là có thể lưu đệm cả kết quả tính toán phức tạp hoặc truy vấn cơ sở dữ liệu trực tiếp, không phụ thuộc vào lệnh `fetch`.

1.3.3. Tác động của việc đọc `cookies()` đến quá trình Render và Caching

Trong kiến trúc Next.js App Router, việc truy cập các Dynamic Functions (như `cookies()`, `headers()`) sẽ tự động chuyển tuyến đường đó sang chế độ **Dynamic Rendering (kết xuất động theo từng yêu cầu)**:

- **Các tuyến đường thực tế đọc cookie trong MovieHub:**
  1. Tuyến `/watchlist` (`app/watchlist/page.tsx`): đọc cookie qua `getWatchlistIds()` để lấy danh sách phim của người dùng.
  2. Tuyến `/movie/[id]` (`app/movie/[id]/page.tsx`): gọi `isMovieSaved(id)` để xác định trạng thái ban đầu của nút lưu phim.
  3. Server Action trong `app/actions/watchlist.ts`: dùng `cookies()` để thêm/xóa phim.
- **Header và MovieCard:** Hai component này không đọc cookie. `Header` dùng `scrollY` và `usePathname`; `MovieCard` nhận phim qua props, không đọc trạng thái Watchlist.

1.3.4. Mối quan hệ tương hỗ giữa Streaming Suspense và Data Caching

Kỹ thuật truyền luồng (Streaming với React Suspense) và Caching dữ liệu (ISR) **không hề loại trừ nhau**, mà hỗ trợ lẫn nhau theo hai giai đoạn rõ rệt:

1. **Khi dữ liệu fetch đã có trong Data Cache:** Các lời gọi TMDB tương ứng có thể dùng lại kết quả; điều này không đồng nghĩa toàn bộ HTML của một route động đã được cache.
2. **Khi dữ liệu chưa có trong cache:** Trang `/movie/[id]` có thể phát phần chính cùng fallback Suspense trước khi `CastList` và `SimilarMovies` hoàn tất. Thời điểm từng chunk phụ thuộc dữ liệu và môi trường chạy, cần Network trace để xác nhận định lượng.

1.3.5. Các cơ chế nền tảng bổ trợ: Hydration, Vòng đời Hook và URL State

Để đảm bảo trải nghiệm người dùng hoàn chỉnh, dự án tích hợp chặt chẽ các cơ chế nền tảng bắt buộc của lập trình web hiện đại:

- **Cơ chế Client Hydration và phòng chống Hydration Mismatch:** Sau khi trình duyệt nhận được HTML render sẵn từ server, React gắn kết các bộ lắng nghe sự kiện (event listeners) vào DOM. Để tránh lỗi bất đồng bộ giữa cây DOM server và client (Hydration Mismatch), các trạng thái phụ thuộc vào trình duyệt (như trạng thái cuộn chuột `window.scrollY` trong `Header`) chỉ được kích hoạt bên trong hook `useEffect` sau khi component đã mount thành công trên client.
- **Quản lý trạng thái thông qua URL (`searchParams`):** Tại các trang `/browse` và `/search`, toàn bộ trạng thái bộ lọc (thể loại, năm, sắp xếp, trang hiện tại) được lưu trữ trực tiếp trên URL Query Parameters. Cách tiếp cận này giúp người dùng có thể chia sẻ liên kết kết quả lọc, lưu bookmark trình duyệt, và sử dụng các nút Back/Forward mà không bị mất trạng thái. Thư viện hook `useTransition` kết hợp `useRouter.replace()` giúp cập nhật URL mượt mà mà không gây đơ cứng giao diện.
- **Điều hướng với `next/link`:** Các liên kết nội bộ như thẻ phim dùng `<Link>` của Next.js để điều hướng phía client. Next.js có thể prefetch tuyến đường đích tùy loại route và cấu hình; thời gian chuyển trang thực tế còn phụ thuộc dữ liệu và mạng.

1.4. Kỹ thuật Streaming UI và React Suspense

1.4.1. Hạn chế của SSR truyền thống (All-or-Nothing Problem)

Trong mô hình Server-Side Rendering truyền thống, quá trình kết xuất trang web mắc phải bài toán thắt nút cổ chai "Tất cả hoặc Không có gì" (All-or-Nothing):

1. Máy chủ bắt buộc phải đợi **tất cả** các truy vấn dữ liệu từ cơ sở dữ liệu và API bên ngoài hoàn thành thì mới có thể bắt đầu tạo tài liệu HTML. Nếu có một API phản hồi chậm (ví dụ: API lấy danh sách diễn viên hoặc phim tương tự mất 800ms), toàn bộ trang web sẽ bị chặn và trình duyệt không hiển thị bất cứ thứ gì.
2. Trình duyệt client phải tải về toàn bộ mã JavaScript của cả trang thì mới có thể tiến hành hydrate.

1.4.2. Cơ chế Streaming HTML lũy tiến

Next.js App Router kết hợp cùng tính năng **React Suspense** của React 19 giải quyết triệt để vấn đề trên thông qua cơ chế truyền luồng (Streaming) [2]. Máy chủ phân rã trang web thành nhiều khối (chunks) độc lập:

- Phần khung chính của trang (Header, thông tin tóm tắt phim) vốn có dữ liệu nhanh sẽ được máy chủ kết xuất và gửi về trình duyệt ngay lập tức cùng với khung giao diện chờ (Skeleton Loader) của phần dữ liệu chậm.

- Thời điểm nội dung chính xuất hiện phải được đo trên bản build production; báo cáo chưa có số đo này.
- Hai Server Components `CastList` và `SimilarMovies` nằm trong các ranh giới `Suspense` riêng, có fallback khi truy vấn còn chờ. Server có thể gửi tiếp phần nội dung hoàn tất qua response đang mở.

1.5. Cơ chế Server Actions và xử lý Mutation dữ liệu

1.5.1. Kiến trúc Remote Procedure Call (RPC)

Để thực hiện thao tác thay đổi dữ liệu từ máy khách, một cách triển khai là tạo API endpoint riêng:

1. Tạo một endpoint API Route riêng biệt (ví dụ: `/api/watchlist`).
2. Viết mã kiểm tra phương thức HTTP (POST/DELETE), đọc tham số từ body.
3. Phía client sử dụng lệnh `fetch('/api/watchlist')` với các bước cấu hình headers phức tạp.

Với **Server Actions**, Next.js giới thiệu mô hình gọi thủ tục từ xa (RPC) tích hợp sâu vào cây component. Lập trình viên chỉ cần khai báo một hàm bất đồng bộ với chỉ thị `"use server"`:

```
// app/actions/watchlist.ts (rut gon)
"use server";
export async function toggleWatchlist(movieId: number) {
  if (!Number.isInteger(movieId) || movieId <= 0) {
    throw new Error("movieId khong hop le");
  }
  const cookieStore = await cookies();
  const currentIds = await getWatchlistIds();
  // Them/bo ID, ghi cookie va revalidatePath("/watchlist")
}
```

Next.js cung cấp lời gọi POST để client kích hoạt Action; hàm vẫn phải tự xác thực dữ liệu đầu vào và kiểm soát quyền truy cập nếu có tài khoản người dùng.

1.5.2. Bảo mật dữ liệu, kiểm thực đầu vào và giới hạn Cookie HttpOnly

Server Actions thực thi hoàn toàn trong môi trường máy chủ bảo mật, mang lại các lợi thế kỹ thuật vượt trội nhưng cũng đòi hỏi các biện pháp bảo vệ đi kèm:

- **Cookie HttpOnly:** `httpOnly: true` chặn truy cập qua `document.cookie`; `secure` được bật trong production để chỉ gửi cookie qua HTTPS. Hai cờ này không tự loại bỏ lỗ hổng XSS hoặc mọi rủi ro bảo mật.

- **Kiểm thực đầu vào:** `toggleWatchlist` kiểm tra `movieId` là số nguyên dương. Tìm kiếm gợi ý qua GET Route Handler xác thực và giới hạn chuỗi truy vấn riêng; nó không phải Server Action.
- **Giới hạn Cookie:** Watchlist giới hạn 200 ID và kiểm tra số byte của chuỗi JSON trước khi ghi. Giới hạn thực tế mỗi cookie tùy trình duyệt; đây là danh sách cục bộ trên một thiết bị, không thay thế cơ sở dữ liệu.
- **Làm mới sau thay đổi:** Action gọi `revalidatePath` cho trang Watchlist và trang chi tiết liên quan. Giao diện cần tải lại dữ liệu server để phản ánh cookie mới.

1.6. Hệ sinh thái công nghệ bổ trợ

1.6.1. Thư viện React 19

Dự án được xây dựng trên nền tảng phiên bản mới nhất **React 19.2.8**. Phiên bản này cung cấp các cải tiến vượt trội về cơ chế quản lý trạng thái tương tác:

- Server Action có thể kết hợp `useActionState` trong ứng dụng khác; MovieHub dùng trạng thái cục bộ tại nút Watchlist.
- Hook `useTransition`: Cho phép đánh dấu các tác vụ cập nhật giao diện là tác vụ không khẩn cấp (non-urgent), giữ cho giao diện luôn phản hồi mượt mà với các thao tác của người dùng.

1.6.2. Framework giao diện Tailwind CSS v4

Dự án áp dụng thế hệ mới nhất **Tailwind CSS v4** [5]. Đây là bước tái kiến trúc toàn diện của framework CSS tiện ích này:

- **Loại bỏ tệp cấu hình truyền thống:** Không còn yêu cầu tệp `tailwind.config.js` phức tạp; toàn bộ các biến giao diện, màu sắc chủ đạo, font chữ được định nghĩa trực tiếp trong tệp CSS nguồn thông qua chỉ thị `@theme`.
- **Engine biên dịch Oxide:** Tailwind CSS v4 cung cấp bộ máy xử lý mới; báo cáo không có benchmark tốc độ giữa v3 và v4 trên repo này.
- **Tương thích tự nhiên với biến CSS (CSS Custom Properties):** Dễ dàng xây dựng hệ thống thiết kế đồng nhất theo tiêu chuẩn điện ảnh Netflix Dark Cinema.

1.6.3. Ngôn ngữ lập trình TypeScript

Mã nguồn của toàn bộ dự án được phát triển 100% bằng **TypeScript** [6] với cấu hình kiểm tra nghiêm ngặt nhất (`"strict": true`). Lập trình viên định nghĩa các giao diện kiểu dữ liệu tường minh (`Movie`, `MovieDetails`, `Cast`, `VideoResponse`) trong thư mục `types/`, ngăn

ngừa triệt để các lỗi truy cập thuộc tính không tồn tại (`undefined`) ngay trong quá trình viết mã.

1.7. Nền tảng dữ liệu mở The Movie Database (TMDB) API

The Movie Database (TMDB) [3] là cơ sở dữ liệu mở phổ biến nhất trên thế giới về phim điện ảnh và chương trình truyền hình. Hệ sinh thái MovieHub khai thác các nhóm dịch vụ REST API v3 của TMDB:

- **Nhóm dữ liệu trang chủ:** `/trending/movie/week`, `/movie/now_playing`, `/movie/top_rated`, `/tv/popular` và `/discover/movie` cho phim kinh điển.
- **Nhóm dữ liệu chi tiết:** `/movie/{id}` cung cấp thông tin phim; các endpoint đuôi `/credits`, `/videos` và `/similar` lần lượt cung cấp diễn viên, video và phim tương tự.
- **Nhóm dữ liệu khám phá và lọc động:** `/discover/movie` và `/discover/tv` dùng tham số thể loại, quốc gia, năm và cách sắp xếp. Với kiểu sắp xếp theo điểm, dịch vụ thêm ngưỡng số phiếu bình chọn.
- **Nhóm dữ liệu phim truyền hình và tìm kiếm:** `/tv/popular` và `/search/multi`.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Khảo sát và đặc tả yêu cầu hệ thống

2.1.1. Yêu cầu chức năng (Functional Requirements)

Hệ thống MovieHub được thiết kế để mang đến trải nghiệm khám phá điện ảnh tương đương các nền tảng thương mại hàng đầu như Netflix, Disney+. Bảng 2.1 tổng hợp đặc tả chi tiết 6 nhóm yêu cầu chức năng cốt lõi.

Bảng 2.1. Đặc tả các yêu cầu chức năng chính của hệ thống MovieHub (Nguồn: Nhóm tác giả đề xuất)

Mã YC	Tên chức năng	Mô tả chi tiết yêu cầu nghiệp vụ
FR01	Duyệt phim Trang chủ	Hero Banner, nút xem trailer và năm hàng phim: thịnh hành, đánh giá cao, đang chiếu, TV phổ biến và phim kinh điển. `MovieRow` hỗ trợ cuộn ngang.
FR02	Tìm kiếm thời gian thực	Thanh tìm kiếm trên Header hỗ trợ Live Search Preview gợi ý tối đa 5 kết quả rút gọn ngay khi người dùng gõ ký tự thông qua Route Handler / Server Action; đồng thời cung cấp trang tìm kiếm độc lập `/search` có cơ chế Debounce.
FR03	Khám phá đa tiêu chí (`/browse`)	`FilterBar` hỗ trợ loại nội dung, thể loại, quốc gia, năm và cách sắp xếp. Phân trang đặt mục tiêu tối đa 24 phim/trang; số thực tế phụ thuộc dữ liệu và lỗi từ TMDB.
FR04	Xem chi tiết phim (`/movie/[id]`)	Hiển thị thông tin tổng quan, thời lượng, năm phát hành, điểm đánh giá TMDB (`vote_average`), dàn diễn viên chính với slider ảnh đại diện tròn, trình phát Trailer YouTube chuẩn điện ảnh, và danh sách phim tương tự đề xuất.
FR05	Quản lý Danh sách yêu thích	Cho phép người dùng đánh dấu hoặc xóa phim khỏi danh sách Watchlist cá nhân mà không cần tài khoản, lưu trữ bền vững qua Cookie `HttpOnly` bảo mật.
FR06	Khám phá Phim bộ (`/tv`)	Cung cấp phân hệ duyệt danh sách phim truyền hình nhiều tập (TV Series) thịnh hành với thông tin số mùa phát sóng.

2.1.2. Yêu cầu phi chức năng (Non-Functional Requirements)

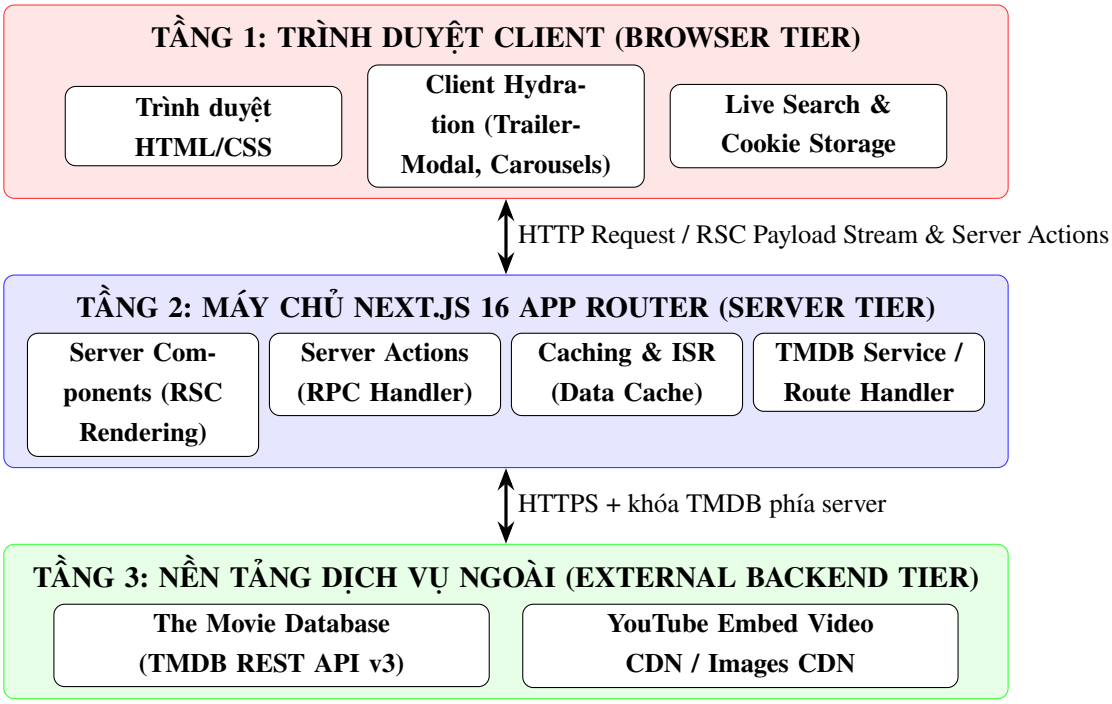
Các tiêu chí thiết kế và khả năng kiểm chứng hiện tại gồm:

- **NFR01 – Hiệu năng:** Ưu tiên Server Components, cache theo thời hạn và ảnh đáp ứng kích thước màn hình. Chưa có dữ liệu Lighthouse để xác nhận ngưỡng LCP.
- **NFR02 – Bố cục:** Khung poster xác định tỉ lệ trước khi tải ảnh; chưa có trace CLS trên trình duyệt để khẳng định giá trị bằng không.
- **NFR03 – Khóa API:** `TMDB_API_KEY` nằm trong biến môi trường phía server; không dùng tiền tố `NEXT_PUBLIC_` và không chuyển khóa qua props.
- **NFR04 – Chịu lỗi:** Các hàm dịch vụ bắt lỗi mạng và trả danh sách rỗng khi phù hợp. Mã DNS dự phòng có giới hạn Node runtime và chưa được kiểm chứng bằng thử nghiệm mạng có kiểm soát.
- **NFR05 – Metadata:** Root layout cung cấp metadata chung; trang chi tiết dùng `generateMetadata` theo phim và ảnh backdrop khi có dữ liệu.

2.2. Kiến trúc hệ thống tổng thể

2.2.1. Mô hình kiến trúc 3 tầng phân lớp

Hệ thống MovieHub được thiết kế theo kiến trúc 3 tầng phân lớp hiện đại (3-Tier Modern Architecture), tận dụng tối đa sức mạnh của Next.js làm tầng trung gian điều phối thông minh. Sơ đồ kiến trúc được minh họa trong Hình 2.1.



Hình 2.1. Sơ đồ kiến trúc 3 tầng phân lớp hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế)

2.2.2. Cấu trúc thư mục dự án theo chuẩn App Router

Mã nguồn dự án được tổ chức khoa học, tách bạch rõ ràng giữa các trang định tuyến, component dùng chung, tầng dịch vụ và định nghĩa kiểu dữ liệu:

- `app/`: Chứa toàn bộ các tuyến đường định tuyến theo cấu trúc thư mục (File-system Routing):
  - `layout.tsx`: Root Layout chứa khung Header và Footer dùng chung toàn ứng dụng.
  - `page.tsx`: Trang chủ hiển thị Hero Banner và các hàng phim chủ đề.
  - `movie/[id]/page.tsx`: Tuyến đường động hiển thị thông tin chi tiết một bộ phim.
  - `browse/page.tsx`: Trang khám phá phim với bộ lọc đa chiều và phân trang số.
  - `tv/page.tsx`: Trang duyệt danh sách các series phim truyền hình.
  - `search/page.tsx`: Trang kết quả tìm kiếm phim có hỗ trợ bộ lọc.
  - `watchlist/page.tsx`: Trang danh sách phim yêu thích cá nhân của người dùng.
  - `actions/watchlist.ts`: Server Action ghi cookie; `api/search/route.ts`: Route Handler GET phục vụ Live Search.
- `components/`: Chứa các UI components được phân chia mô-đun hóa:

  - `Header.tsx`, `NavbarDropdown.tsx`, `NavSearchBar.tsx`: Các thành phần thanh điều hướng cao cấp.
  - `HeroBanner.tsx`, `TrailerModal.tsx`: Banner chính và modal xem video trailer.
  - `MovieRow.tsx`, `CastSlider.tsx`, `SimilarMoviesSlider.tsx`: Các thanh cuộn carousel mượt mà chống kẹt biên.
  - `FilterBar.tsx`: Thanh công cụ lọc đa chiều linh hoạt.
  - `MovieCard.tsx`: Thẻ hiển thị áp phích phim chuẩn kích thước.
- `services/client.ts`: Hàm `fetchTMDB`, cấu hình DNS và khử trùng lặp; `movies.ts`, `discover.ts`, `search.ts`, `videos.ts` chứa các lời gọi theo nghiệp vụ; `tmdb.ts` tái xuất các hàm cho trang.
- `types/`: Chứa toàn bộ các định nghĩa kiểu dữ liệu TypeScript.

2.3. Thiết kế tích hợp dịch vụ và luồng dữ liệu

2.3.1. Kiến trúc tầng dịch vụ `services/client.ts`

Tất cả các lời gọi mạng ra ngoài TMDB đều được tập trung vào hàm cốt lõi `fetchTMDB<T>()`. Hàm này chịu trách nhiệm:

1. Dùng URL mặc định `https://api.themoviedb.org/3`; có thể thay bằng `TMDB_BASE_URL` để kiểm thử.
2. Đính kèm `api_key=${TMDB_API_KEY}` ở yêu cầu từ server tới TMDB; không dùng tiền tố `NEXT_PUBLIC_`. Cần tránh ghi URL có khóa vào log hoặc chuyển nó qua props.
3. Kiểm soát lỗi mạng và mã trạng thái HTTP, trả về thông báo lỗi chuẩn xác hoặc kích hoạt cơ chế dự phòng khi có sự cố.

2.3.2. Giải pháp kiến trúc DNS Fallback Resolver

Lỗi phân giải tên miền `ENOTFOUND` là một tình huống mạng có thể xảy ra khi truy cập TMDB. Dự án có mã thử nghiệm DNS dự phòng, nhưng chưa lưu log đối chứng để quy nguyên nhân cho ISP cụ thể.

`services/client.ts` có mã thay `dns.lookup` toàn cục tại Node runtime và thử phân giải TMDB qua `Resolver`. Giải pháp này cần được thử cô lập trước khi coi là cơ chế chịu lỗi đáng tin cậy:

- Khởi tạo một đối tượng phân giải độc lập `dns.promises.Resolver()` với danh sách máy chủ DNS công cộng quốc tế uy tín: Google DNS (`8.8.8.8`) và Cloudflare DNS (`1.1.1.1`).

- Ghi đè (hook/override) hàm `dns.lookup` của Node.js: Khi phát hiện yêu cầu phân giải tên miền chứa `themoviedb.org`, hệ thống sẽ sử dụng `resolver.resolve4()` để lấy trực tiếp địa chỉ IPv4 từ Google/Cloudflare, bỏ qua DNS mặc định của nhà mạng.
- **Phạm vi và giới hạn:** Chỉ chạy trên Node runtime; chưa chứng minh `fetch` luôn sử dụng hook này. Mã có thể ảnh hưởng các lời gọi DNS khác trong cùng tiến trình nên cần kiểm tra thêm trước khi triển khai production.

2.3.3. Phân tầng chiến lược Caching và Revalidation

Bảng 2.2 ghi thời hạn `next.revalidate` trong các hàm dịch vụ; đây là cấu hình Data Cache cho yêu cầu mạng, không mặc nhiên là thời hạn ISR của mọi route.

Bảng 2.2. Thời hạn revalidate của các yêu cầu TMDB trong tầng services

Nhóm dữ liệu	**Chiến lược Cache**	**Thời gian TTL**	**Mục đích và Lý do thiết kế**
Phim thịnh hành và TV phổ biến	`fetch` Data Cache	3.600s	`getTrendingMovies`, `getTvPopular`.
Phim đang chiếu / điểm cao	`fetch` Data Cache	1.800s / 86.400s	`getNowPlayingMovies`, `getTopRatedMovies`.
Chi tiết, diễn viên, phim tương tự	`fetch` Data Cache	3.600s	Route chi tiết vẫn động vì đọc cookie; các phần phụ dùng Suspense.
Khám phá / tìm kiếm	`fetch` Data Cache	1.800s / 300s / 120s	Khám phá, tìm kiếm trang đầy đủ và Live Search lần lượt dùng các thời hạn này.

2.4. Thiết kế giao diện và trải nghiệm người dùng (UI/UX)

2.4.1. Hệ thống thiết kế chuẩn điện ảnh (Netflix Dark Cinema Design System)

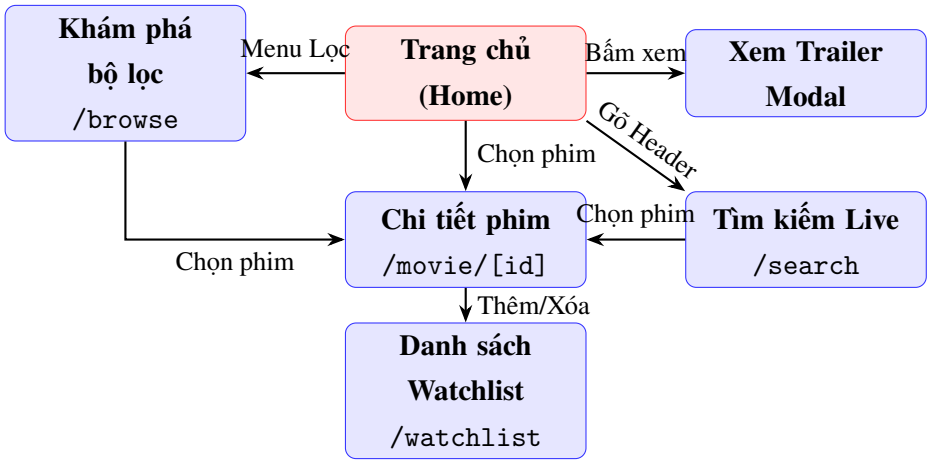
Để mang lại trải nghiệm xem phim đẳng cấp và chân thực, giao diện MovieHub được xây dựng tuân thủ nghiêm ngặt theo hợp đồng thiết kế `DESIGN.md`. Bảng 2.3 liệt kê các thông số thiết kế chủ đạo.

Bảng 2.3. Bảng mã màu và thông số thiết kế Netflix Dark Cinema (Nguồn: Trích xuất từ tài liệu DESIGN.md của dự án)

Yếu tố thiết kế	Giá trị mã màu	Mục đích và Vị trí áp dụng
Màu nền chủ đạo	`#141414`	Tạo chiều sâu điện ảnh tối đa, làm nổi bật màu sắc rực rỡ của các áp phích phim; giảm mỏi mắt cho người dùng trong điều kiện thiếu sáng.
Màu đỏ thương hiệu	`#E50914`	Điểm nhấn thị giác cho Logo thương hiệu, các nút kêu gọi hành động (CTA), viền active của danh mục và chỉ báo đang xem.
Màu bề mặt phụ	`#2F2F2F`	Sử dụng cho thanh điều hướng phụ, các thẻ card khi hover và nền khung skeleton chờ.
Màu văn bản chính	`#FFFFFF`	Chữ trắng trên nền xám đen (`#141414`) có tỉ lệ tương phản khoảng 18,7:1 theo công thức WCAG; đây chỉ là kiểm tra cặp màu, không phải chứng nhận toàn trang đạt WCAG.
Màu văn bản phụ	`#AAAAAA`	Sử dụng cho ngày phát hành, thông số thời lượng, tóm tắt nội dung phụ.

2.4.2. Sơ đồ điều hướng và luồng trải nghiệm người dùng

Hình 2.2 thể hiện sơ đồ luồng người dùng (User Navigation Flow) khi tương tác với các phân hệ chức năng của ứng dụng.



Hình 2.2. Sơ đồ luồng điều hướng trải nghiệm người dùng trong hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế)

2.5. Môi trường và công cụ phát triển

Dự án được triển khai trên nền tảng phần mềm hiện đại và ổn định:

- **Môi trường kiểm chứng (Runtime):** Node.js v24.19.0 tại máy chạy lệnh; dự án dùng API web stream và module DNS của Node.js.
- **Khung ứng dụng (Framework):** Next.js phiên bản 16.3.6 (App Router) kết hợp thư viện React 19.2.8.
- **Ngôn ngữ và Công cụ quản lý gói:** TypeScript 5.x và npm (Node Package Manager).
- **Môi trường lập trình:** Visual Studio Code tích hợp tiện ích mở rộng ESLint, Tailwind CSS IntelliSense và GitLens.
- **Công cụ kiểm chứng:** ESLint, TypeScript, Node test runner và `next build`; dự án có cấu hình analyzer nhưng chưa lưu kết quả đo.
- **Công cụ biên soạn tài liệu kỹ thuật:** Hệ thống MiKTeX với trình biên dịch `pdflatex`.

CHƯƠNG 3. HIỆN THỰC HỆ THỐNG

3.1. Cài đặt và cấu hình dự án

Dự án được khởi tạo bằng công cụ chính thống của Vercel với cấu hình TypeScript và Tailwind CSS:

```
npx create-next-app@latest moviehub --typescript --tailwind --app --
    eslint
```

Để bảo mật các khóa truy cập dữ liệu, tệp cấu hình môi trường `.env.local` được thiết lập độc quyền trên máy chủ:

```
TMDB_API_KEY=your_tmdb_api_key_here
TMDB_BASE_URL=https://api.themoviedb.org/3
TMDB_IMAGE_BASE_URL=https://image.tmdb.org/t/p
```

Khóa TMDB không có tiền tố `NEXT_PUBLIC_`; tầng dịch vụ gọi TMDB từ server. Không đưa khóa vào client props hoặc URL của trang web.

Thông tin phiên bản các gói phần mềm cốt lõi được cấu hình tường minh trong `package.json` của dự án:

```
{
  "name": "modern-programming-technologies",
  "version": "0.1.0",
  "dependencies": {
    "next": "16.3.6",
    "react": "19.2.8",
    "react-dom": "19.2.8"
  },
  "devDependencies": {
    "@next/bundle-analyzer": "^16.3.6",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

Lệnh `npx next -version` xác nhận phiên bản `Next.js v16.3.6`; phiên bản `React v19.2.8` được xác nhận từ `package.json` và `npm ls react -depth=0`.

3.2. Hiện thực Tầng 1A: Các tính năng cốt lõi bắt buộc

3.2.1. Phân định ranh giới Server/Client Component

Ứng dụng thực hiện phân tách ranh giới rõ ràng:

- **Root Layout** (`app/layout.tsx`)**:** Đóng vai trò là Server Component gốc, chịu trách nhiệm nhập bộ font chữ chuẩn, cấu hình thẻ `<html>`, `<body>` và bao bọc toàn bộ khung ứng dụng bằng thanh điều hướng `Header` và chân trang `Footer`.
- **Trang chủ** (`app/page.tsx`)**:** Đóng vai trò là Server Component điều phối, chịu trách nhiệm thu thập toàn bộ các danh sách phim song song và truyền dữ liệu dạng props tĩnh xuống các component con.

3.2.2. Server-side Data Fetching và song song hóa với `Promise.all`

Trang chủ `app/page.tsx` là Server Component bất đồng bộ. Năm lời gọi dữ liệu được khởi tạo cùng lúc; kết quả được truyền vào giao diện sau khi tất cả hoàn tất. Dữ liệu TMDB được lấy bởi `fetchTMDB` trong `services/client.ts`, sử dụng biến môi trường `TMDB_API_KEY` và `next: { revalidate }`. Khóa API không có tiền tố `NEXT_PUBLIC_`.

```
// Trich tu app/page.tsx
const [trending, nowPlaying, topRated, tvData, classicData] =
  await Promise.all([
    getTrendingMovies(), getNowPlayingMovies(),
    getTopRatedMovies(), getTvPopular(),
    discoverMovies({ genre: "classic" }),
  ]);
```

Thời gian phản hồi thực tế phụ thuộc mạng và trạng thái cache; báo cáo không suy ra một giá trị mili giây cố định từ cấu trúc `Promise.all`. Các `MovieCard` được tạo trong trang server và truyền vào `MovieRow` bằng prop `cards`; mã của thanh cuộn vẫn ở client để xử lý thao tác nút mũi tên.

3.2.3. Dynamic Routing (`/movie/[id]`) và Dynamic SEO Metadata

Trang chi tiết phim áp dụng quy ước định tuyến động `app/movie/[id]/page.tsx`. Kể từ Next.js 15, tham số `params` đã chuyển sang dạng Promise bất đồng bộ và trở thành quy chuẩn bắt buộc trong Next.js 16, yêu cầu xử lý chuẩn xác bằng cú pháp `await`:

```
// app/movie/[id]/page.tsx
interface PageProps {
  params: Promise<{ id: string }>;
}

export async function generateMetadata({ params }: PageProps): Promise
    <Metadata> {
  const { id } = await params;
    const movie = await getMovieDetails(id);
    const year = movie.release_date ? movie.release_date.split("-")[0]
    : "";
```

```
  return {
    title: `${movie.title} ${year ? `(${year})` : ""} | MovieHub`,
    description: movie.overview || `Thong tin phim ${movie.title}`,
    openGraph: {
      title: movie.title,
      description: movie.overview,
      images: movie.poster_path
        ? [`https://image.tmdb.org/t/p/w500${movie.poster_path}`]
        : [],
    },
  };
}
```

3.3. Hiện thực Tầng 1B: Các kỹ thuật chuyên sâu và nâng cao

3.3.1. Server Action cho mutation và Route Handler cho tìm kiếm

Tác vụ thay đổi dữ liệu dùng `toggleWatchlist` và `clearAllWatchlist` trong `app/actions/watchlist.ts`. Action kiểm tra ID phim, ghi cookie `HttpOnly`, `SameSite=Lax`, bật `Secure` trong production rồi gọi `revalidatePath` cho trang Watchlist.

Live Search là tác vụ đọc. `NavSearchBar` đợi 300ms, gọi GET `/api/search` và hủy yêu cầu cũ bằng `AbortController`. Route Handler trong `app/api/search/route.ts` xác thực từ khóa rồi gọi `searchQuickLive` trên server. Header không gọi Server Action cho từng phím. Tiêu đề `s-maxage=120` hướng tới shared cache/CDN; mức cache thực tế cần đo trên môi trường triển khai.

3.3.2. Streaming Suspense và Skeleton Loading States

Tại `app/movie/[id]/page.tsx`, thông tin chính cần `getMovieDetails`, `getMovieVideos` và trạng thái watchlist trước khi phần thân trang được dựng. Hai khối dữ liệu phụ `CastList` và `SimilarMovies` nằm trong các ranh giới Suspense độc lập với fallback tương ứng. Mã thật sử dụng `CastSkeleton` và một dãy khung poster chờ:

```
<Suspense fallback={<CastSkeleton />}>
  <CastList movieId={id} />
</Suspense>
<Suspense fallback={<div className="flex gap-3 overflow-hidden">
  {Array.from({ length: 6 }).map((_, i) => (
    <div key={i} className="w-36 aspect-[2/3] bg-zinc-800" />
  ))}
</div>}>
  <SimilarMovies movieId={id} />
```

```
</Suspense>
```

Đoạn mã rút gọn lớp CSS của fallback thứ hai nhưng giữ cấu trúc JSX thực tế. Suspense cho phép gửi phần đã sẵn sàng trước dữ liệu phụ khi truy vấn phụ chậm; thời điểm byte đầu và thời điểm vẽ nội dung cần đo riêng, không thể suy ra từ cấu trúc JSX.

3.3.3. Tối ưu hóa hình ảnh với `next/image` và YouTube Player API

Các ảnh phim trong các component chính dùng `next/image`:

- `MovieCard` dùng `sizes="(max-width: 640px) 40vw, (max-width: 1024px) 25vw, 15vw"`; Hero Banner dùng `sizes="100vw"`.
- Khung poster có tỉ lệ 2:3 để dành chỗ trước khi ảnh tải xong; hiệu quả CLS thực tế cần đo trên trình duyệt.
- `TrailerModal` nhúng YouTube bằng `iframe` với `autoplay=1`; khả năng tự phát còn phụ thuộc chính sách của trình duyệt.

3.4. Hiện thực các module giao diện người dùng chính

3.4.1. Module Hero Banner & Trailer Modal xem trailer tức thì

Component `TrailerModal.tsx` là Client Component. Khi người dùng bấm "Xem Trailer", modal mở trên giao diện; phím `Escape` hoặc nhấp bên ngoài đóng modal. Nếu TMDB không có trailer, component dùng liên kết tìm kiếm YouTube. Hàm chọn trailer nằm trong `services/videos.ts`.

3.4.2. Module MovieRow Slider & Carousel cuộn vô tận

Component `MovieRow.tsx` mang lại trải nghiệm cuộn phim đặc trưng của Netflix:

- Tích hợp hai nút điều hướng mũi tên trái và phải, tự động ẩn hiện thông minh khi người dùng rê chuột vào hàng phim.
- Áp dụng thuật toán kiểm tra biên dính (`wasStuck`): Khi cuộn đến điểm cuối của danh sách, hệ thống tự động tính toán để chuyển hướng mượt mà, tạo cảm giác danh sách nối tiếp không giới hạn.
- Hỗ trợ thao tác vuốt cảm ứng (touch swipe) tự nhiên trên các thiết bị máy tính bảng và điện thoại di động.

3.4.3. Module FilterBar & Trang Khám phá `/browse`

Trang `/browse` cung cấp bộ công cụ tìm kiếm điện ảnh chuyên sâu:

- Thanh công cụ `FilterBar` cho phép kết hợp đồng thời 5 tiêu chí độc lập: (1) Định dạng nội dung (`type`: Phim lẻ, Phim bộ, Tất cả), (2) Thể loại (`genre`: 19 thể loại phim TMDB), (3) Quốc gia (`country`: 7 quốc gia chọn lọc), (4) Năm phát hành (`year`: từ 2026 trở về trước), và (5) Sắp xếp (`sortBy`: độ phổ biến, điểm số, ngày ra mắt, doanh thu).
- **Phân trang:** `services/discover.ts` ghép các trang TMDB và loại ID trùng để lấy tối đa 24 phim cho mỗi trang UI. Kết quả có thể ít hơn khi dữ liệu nguồn không đủ hoặc yêu cầu mạng lỗi.

3.4.4. Module Thanh điều hướng nâng cao (`NavbarDropdown`, `NavSearchBar`)

Thanh điều hướng `Header` sở hữu các tính năng cao cấp:

- **Hiệu ứng thanh điều hướng:** Ở các trang có Hero Banner, Header bắt đầu trong suốt và chuyển sang nền `#141414` khi `window.scrollY > 30`.
- **Mega Menu Dropdown (`NavbarDropdown.tsx`):** Hiển thị danh sách đầy đủ 19 thể loại phim TMDB dưới dạng lưới 2 cột tiện lợi.
- **Live Search Bar (`NavSearchBar.tsx`):** Hộp tìm kiếm nhanh hiển thị kết quả xem trước ngay bên dưới thanh điều hướng, cho phép nhấp vào để chuyển thẳng tới trang chi tiết phim.

3.4.5. Module Trang Phim bộ (`/tv`) và Khám phá diễn viên

Phân hệ `/tv` được thiết kế tương thích với các chương trình truyền hình nhiều mùa; đồng thời các component `CastSlider` và `SimilarMoviesSlider` cung cấp cái nhìn toàn diện về đội ngũ sản xuất và các tác phẩm điện ảnh cùng phong cách.

3.5. Hiện thực cơ chế tự phục hồi và khử trùng lặp dữ liệu

3.5.1. Giới hạn của cơ chế DNS dự phòng

`services/client.ts` chứa một bộ `Resolver` dùng Google DNS và Cloudflare cho tên miền TMDB, đồng thời thay `dns.lookup` ở tiến trình Node.js. Đây là thử nghiệm xử lý hạ tầng, không phải cơ chế của Next.js. Giải pháp thay đổi hàm DNS toàn cục và phụ thuộc Node runtime; báo cáo chưa có log tái lập tình huống ISP chặn DNS hoặc phép kiểm tra chứng minh mọi đường gọi `fetch` đều đi qua hook này. Vì vậy nhóm không xem DNS fallback là minh chứng hoàn thành Tầng 1B hay kết luận có thể tự phục hồi trong mọi mạng.

3.5.2. *Thuật toán khử trùng lặp phim (`deduplicateMovies`)*

Khi ghép các trang TMDB, phim có thể lặp lại ở ranh giới trang. Hàm `deduplicateMovies` trong `services/client.ts` dùng `Set` để giữ phần tử đầu theo ID; test tương ứng nằm trong `tests/deduplicate.test.mjs`:

```
function deduplicateMovies(movies: Movie[]): Movie[] {
  const seen = new Set<number>();
  const unique: Movie[] = [];
  for (const movie of movies) {
    if (movie && movie.id && !seen.has(movie.id)) {
      seen.add(movie.id);
      unique.push(movie);
    }
  }
  return unique;
}
```

3.6. Tổ chức phát triển và tiến độ thực hiện

3.6.1. *Bảng phân công trách nhiệm nhóm*

Bảng 3.1 ghi phân công và tỷ lệ nhóm tự khai. Lịch sử Git cho thấy các phần được tích hợp qua Pull Request, nhưng phần trăm đóng góp không được suy ra bằng thống kê số dòng code.

Bảng 3.1. Bảng phân công trách nhiệm và tỷ lệ đóng góp của các thành viên (Nguồn: Nhóm tác giả thống nhất)

Họ và tên	Mã số SV	Nhiệm vụ kỹ thuật đảm nhiệm chính	Đóng góp
Nguyễn Công Quân (Trưởng nhóm)	3123560070	Theo phân công nhóm: kiến trúc nền, trang chủ, tầng dịch vụ TMDB, MovieRow, TrailerModal và tổng hợp báo cáo. Các chỉ số hiệu năng chưa có tệp đo để quy về cá nhân.	33.3%
Nguyễn Quốc Đại	3123560013	Theo phân công nhóm: route chi tiết phim, hai vùng Suspense, slider diễn viên và phim tương tự, giao diện chờ/lỗi và metadata động. Chưa có tệp trace TTFB để đưa ra số đo.	33.3%
Nguyễn Thái Vinh	3123410433	Theo phân công nhóm: trang tìm kiếm, Watchlist Action, Browse/FilterBar, trang TV và Header/Live Search dùng GET Route Handler. Chưa có tệp bundle analyzer để đưa ra số đo.	33.3%

3.6.2. Tiến độ thực hiện theo tuần (Tóm tắt kế hoạch 7 tuần)

Kế hoạch học phần kéo dài 7 tuần. Bảng 3.2 đối chiếu các nhóm công việc với hiện vật mã nguồn; bảng không thay thế nhật ký sprint hay biên bản nghiệm thu theo tuần.

Bảng 3.2. Các nhóm công việc trong kế hoạch 7 tuần và hiện vật tương ứng

Giai đoạn	Mục tiêu trọng tâm	Kết quả kỹ thuật nghiệm thu
Tuần 1	Khởi tạo và thiết lập kiến trúc	Có App Router, TypeScript strict, Tailwind CSS v4 và tầng dịch vụ TMDB trong `services/`.
Tuần 2	Giao diện cốt lõi & Data Fetching	Có Root Layout, Header và trang chủ với năm nhóm dữ liệu gọi song song; ảnh chính dùng `next/image`.
Tuần 3	Dynamic Routing & Chi tiết phim	Hiện thực tuyến đường động `/movie/[id]`; xây dựng giao diện chi tiết, hiển thị backdrop, thông tin doanh thu, thời lượng; thiết lập hàm sinh siêu dữ liệu động `generateMetadata`.
Tuần 4	Kỹ thuật cốt lõi Streaming & Actions	Triển khai React Suspense Streaming cho dàn diễn viên và phim tương tự; xây dựng Server Actions quản lý Watchlist với Cookie `HttpOnly`; hoàn thiện bộ ba trang `loading`, `error`, `not-found`.
Tuần 5	Tìm kiếm Debounce & Bộ lọc	Xây dựng trang tìm kiếm `/search` có cơ chế Debounce; khởi dựng trang khám phá `/browse` kết nối endpoint TMDB Discover; hoàn thiện cơ chế cuộn mượt cho các hàng phim.
Tuần 6	Hoàn thiện tính năng nâng cao	Lập trình component `TrailerModal.tsx` phát YouTube trailer nổi; hoàn thiện bộ lọc `FilterBar` 5 chiều và phân trang số 24 phim; xây dựng Mega Menu Dropdown và Live Search trên Header; tích hợp cơ chế DNS Fallback Resolver.
Tuần 7	Kiểm chứng và báo cáo	Chạy lint, type-check, ba test Node và production build; tổng hợp ma trận yêu cầu. Chưa có kết quả Lighthouse, bundle analyzer hoặc trace TTFB gốc.

3.6.3. Quy trình làm việc nhóm với Git Flow

Dự án dùng Git và có lịch sử tích hợp qua Pull Request:

- Bản mã nguồn đang kiểm tra đã chạy production build. Chưa có bằng chứng trong repo về cấu hình bảo vệ nhánh `main` trên GitHub.
- Các nhánh tính năng trong lịch sử gồm nhóm trang chủ, trang chi tiết, tìm kiếm/Watchlist và Browse/Filter.
- Lịch sử Git cục bộ có 7 merge commit ghi "Merge pull request". Repo hiện có workflow CI cho lint, type-check, test và build; báo cáo chưa xác minh quy tắc review bắt

buộc của từng PR trên GitHub.

CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Phương pháp và giới hạn của bằng chứng

Nhóm đánh giá hai loại bằng chứng: (1) mã nguồn và lệnh kiểm tra có thể chạy lại; (2) kết quả vận hành cần môi trường và dữ liệu đo cụ thể. Các lệnh dùng cho bản mã nguồn hiện tại là `npm run lint`, `npx tsc -noEmit`, `npm test` và `npm run build`. Log build được lưu tại `baocao/evidence/build-output.txt`, log test ở `baocao/evidence/test-output.txt`. Lệnh build dùng Next.js 16.3.6/Turbopack; thông tin route ở Bảng 4.2 lấy từ log build production.

Bản báo cáo này chưa có tệp xuất Lighthouse, trace trình duyệt, kết quả analyzer trước/sau hoặc dữ liệu thô 10 lần đo TTFB. Vì vậy nhóm **không công bố** điểm Performance, LCP, FCP, CLS, dung lượng bundle hoặc tỷ lệ cải thiện streaming. Những chỉ số đó chỉ nên đưa vào bản nộp khi có cấu hình thiết bị, URL, môi trường production, lần đo và tệp kết quả đi kèm. TTFB là thời điểm byte đầu về client; FCP là thời điểm trình duyệt vẽ nội dung, hai chỉ số không thể thay thế nhau.

4.2. Kiểm tra tự động và khả năng chạy lại

Bảng 4.1. Các kiểm tra có thể thực thi lại trên mã nguồn hiện tại

Lệnh	Phạm vi xác minh	Kết quả
`npm run lint`	Quy tắc ESLint của dự án Next.js.	Exit code 0
`npx tsc -noEmit`	Kiểu dữ liệu TypeScript ở chế độ `strict`.	Exit code 0
`npm test`	Ba test Node: khử trùng lặp ID, ranh giới module Server/Client, và cấu hình yêu cầu TMDB với khóa giả cùng thời hạn revalidate.	3/3 đạt
`npm run build`	Biên dịch production, kiểm tra route tĩnh/động và chu kỳ tái tạo của trang chủ.	Exit code 0

Tệp `.github/workflows/check.yml` cấu hình chạy các lệnh trên khi push hoặc mở Pull Request; `README.md` và `.env.example` hướng dẫn cấu hình khóa TMDB. Workflow đọc `TMDB_API_KEY` từ GitHub secret; báo cáo chưa có log một lượt chạy CI để xác nhận kết quả trên GitHub. Ba test cục bộ dùng khóa giả để xác minh logic, ranh giới module và cấu hình server fetch; chúng không thay thế kiểm thử giao diện trên trình duyệt. Log GET production tại localhost nằm ở `baocao/evidence/runtime-http.txt`.

4.3. Bằng chứng về mô hình render và cache

Bảng 4.2. Phân loại route từ `npm run build` trên bản mã nguồn hiện tại

Route	Kết quả	Giải thích từ mã nguồn
/	Tĩnh, revalidate 30 phút	app/page.tsx không đọc request-time API; thời hạn thấp nhất trong năm nhóm fetch là 1800 giây.
/_not-found	Tĩnh	Trang lỗi mặc định được tạo trước.
/api/search	Động	Route Handler đọc query từ request.
/browse	Động	Đọc searchParams để lọc.
/movie/[id]	Động	Đọc tham số id và cookie Watchlist; có hai ranh giới Suspense.
/search	Động	Đọc searchParams.q.
/tv	Động	Đọc searchParams.page.
/watchlist	Động	Đọc await cookies() theo người dùng.

Route phim không hợp lệ. scripts/verify-runtime-http.py gọi một ID lấy từ HTML trang chủ và một ID không hợp lệ. ID thật trả HTTP 200; ID sai hiển thị giao diện not-found.tsx và thẻ robots=noindex. Với response streaming, Next.js có thể đã gửi mã HTTP 200 trước khi notFound() được kích hoạt; đây là "soft 404" theo tài liệu not-found của Next.js [1], không phải bằng chứng HTTP status 404. Nếu yêu cầu bắt buộc trả status 404, cần kiểm tra tài nguyên trước khi stream.

Ranh giới Server/Client tại trang chủ. app/page.tsx gọi TMDB qua tầng dịch vụ trên server, tạo MovieCard rồi truyền các phần tử React trong prop cards cho MovieRow. Thanh cuộn chỉ xử lý cuộn và nút điều hướng; test tests/server-client-boundary.test.mjs kiểm tra nó không nhập MovieCard vào module graph phía client. Script scripts/verify-render-evidence.py đọc HTML được prerender ở .next/server/app/index.html sau build. Log baocao/evidence/server-render.txt ghi 5/5 tiêu đề hàng phim, 107 thẻ ảnh có alt trong HTML và không thấy chuỗi api_key= trong tệp HTML đó. Log này chứng minh nội dung trang chủ xuất hiện trong HTML build; nó không phải kiểm tra toàn bộ lưu lượng mạng hoặc phép đo kích thước bundle.

Caching. fetchTMDB gọi fetch với next.revalidate; test tests/server-fetch.test.mjs chặn fetch để xác nhận URL server chứa khóa giả và tùy chọn revalidate: 1800. Đây là cấu hình Data Cache cho request TMDB, khác với việc route / được tạo tĩnh và tái tạo sau 30 phút. cacheComponents không bật. Muốn chứng minh cache hit/miss định lượng, cần ghi số request upstream trước và sau thời hạn; test cấu hình không phải số đo hit/miss.

Streaming. Trang chi tiết phải chờ dữ liệu phim, video và trạng thái Watchlist trước phần nội dung chính. Sau đó CastList và SimilarMovies có thể stream độc lập khi truy vấn của

chúng hoàn tất. `loading.tsx` cung cấp giao diện chờ khi điều hướng. Để nghiệm thu tương tác, mở trang chi tiết trên bản production với mạng chậm, ghi video hoặc Network waterfall chứng minh fallback xuất hiện trước nội dung của hai khối này; báo cáo hiện chỉ chứng minh cấu trúc trong code, không tuyên bố mức giảm TTFB.

Mutation và tác vụ đọc. `toggleWatchlist` ghi cookie sau khi kiểm tra `movieId`, giới hạn 200 ID và làm mới `/watchlist`; `NavSearchBar` dùng Route Handler GET cho gợi ý tìm kiếm. Các thao tác này phải được thử với trạng thái trước/sau trên trình duyệt khi bảo vệ. Cookie `HttpOnly` chỉ ngăn JavaScript đọc trực tiếp qua `document.cookie`; nó không loại bỏ mọi rủi ro XSS.

4.4. Ma trận đáp ứng yêu cầu môn học

Bảng 4.3. Đối chiếu yêu cầu và tệp minh chứng trong repo

Tầng	Tiêu chí	Hiện thực và minh chứng
1A	App Router, page/layout, next/link	`app/layout.tsx`, `app/page.tsx`; liên kết ở `MovieCard` và `MovieRow`. Bảng 4.2 là output build.
1A	Server/Client, props, state, hydration	Trang chủ dựng card ở server; `MovieRow` nhận props và xử lý sự kiện; `FilterBar` dùng URL state; `Header` dùng `useEffect` cho scroll.
1A	Data fetching trên server	`services/client.ts` chứa `fetchTMDB`; `app/page.tsx` gọi năm nhóm dữ liệu song song. Log HTML build: `baocao/evidence/server-render.txt`.
1A	API bất đồng bộ của Next.js 16	`app/movie/[id]/page.tsx` dùng `await params`; `app/browse/page.tsx` dùng `await searchParams`; Watchlist dùng `await cookies()`.
1B	Route động, loading/error/not-found	`app/movie/[id]/page.tsx`, `loading.tsx`, `error.tsx`, `not-found.tsx`; log `baocao/evidence/runtime-http.txt` xác nhận UI không tìm thấy và `noindex`.
1B	Cache, revalidation	`services/client.ts` với `next.revalidate`; `baocao/evidence/build-output.txt` xác nhận `/` tĩnh, 30 phút. Chưa có đo cache hit/miss.
1B	Server Action và Route Handler	`app/actions/watchlist.ts` xử lý ghi cookie; `app/api/search/route.ts` xử lý GET.
1B	Suspense, image, font, metadata	`app/movie/[id]/page.tsx`; `MovieCard.tsx` dùng `next/image`; `app/layout.tsx` dùng `next/font`; `next.config.ts` có `remotePatterns`; `generateMetadata` theo phim.

Tầng	Tiêu chí	Hiện thực và minh chứng
2	Git, chạy lại, cấu hình bí mật	Lịch sử 7 merge PR trong Git; `README.md`, `.env.example`, `package.json`. Không đưa khóa thật vào báo cáo.
2	Test và CI	`tests/*.test.mjs`, script `npm test`, `.github/workflows/check.yml`; log `baocao/evidence/test-output.txt`.
2	Bảo mật và xử lý lỗi	Xác thực đầu vào ở Route Handler/Server Action; `app/movie/[id]/error.tsx` và `not-found.tsx`; giới hạn của DNS fallback được nêu ở Chương 3.
3	Đo hiệu năng nâng cao	Chưa đủ dữ liệu gốc cho Lighthouse, bundle hoặc TTFB đối chứng; không đăng ký các chỉ số đó là kết quả đã nghiệm thu.

Ma trận nêu đúng những gì có thể xác minh từ repo. Các mục cần quan sát trực tiếp (fallback streaming, phản hồi Watchlist và cache hit/miss) vẫn cần ảnh, video hoặc trace khi nghiệm thu; nhóm không thay chúng bằng số liệu suy đoán.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được. MovieHub minh họa được các cơ chế chính của Next.js 16 trên một nghiệp vụ đơn giản: App Router, Server Components lấy dữ liệu TMDB, Client Components cho thao tác giao diện, route động với tham số bất đồng bộ, Data Cache có thời hạn, Streaming Suspense, metadata động, Server Action cho Watchlist và Route Handler GET cho tìm kiếm. Bản build production phân loại trang chủ là route tĩnh tái tạo sau 30 phút, còn các route phụ thuộc query, ID hoặc cookie là động. Lint, kiểm tra kiểu, ba test tự động và build production đã được chạy trên bản mã nguồn hiện tại; CI cấu hình chạy lại các lệnh đó khi cập nhật repo.

2. Giới hạn được nhận diện. Việc truyền phần tử `MovieCard` từ server vào các thanh cuộn client làm rõ ranh giới module, nhưng báo cáo chưa có phép đo bundle trước/sau để lượng hóa lợi ích. Tương tự, mã có Suspense và revalidation nhưng chưa có trace/ảnh và dữ liệu thô đủ để khẳng định mức cải thiện TTFB, Core Web Vitals hay tỉ lệ cache hit. Bộ test hiện tại kiểm tra logic và kiến trúc, chưa có kiểm thử E2E trên trình duyệt. Watchlist lưu trong cookie bị giới hạn dung lượng và không đồng bộ giữa thiết bị. Mã DNS dự phòng chỉ dùng Node runtime, can thiệp hàm toàn cục và chưa có thử nghiệm cô lập chứng minh hiệu quả trong mọi môi trường mạng.

3. Hướng hoàn thiện. Nhóm ưu tiên ba việc có thể nghiệm thu rõ ràng: (1) lưu video/Network waterfall của trang chi tiết và thao tác Watchlist trước/sau; (2) ghi số request TMDB khi cache miss, cache hit và sau thời hạn revalidate; (3) xuất báo cáo Lighthouse và bundle analyzer kèm phiên bản trình duyệt, cấu hình đo, URL và dữ liệu thô. Chỉ sau đó mới đưa các chỉ số định lượng vào kết luận. Về sản phẩm, có thể thay Watchlist cookie bằng cơ sở dữ liệu và cơ chế xác thực nếu cần đồng bộ nhiều thiết bị.

TÀI LIỆU

[1] Vercel Inc., "Next.js Documentation: App Router, Server Components and Data Fetching," 2026. [Trực tuyến]. Địa chỉ: https://nextjs.org/docs. [Truy cập: 15-09-2026].

[2] React Core Team, "React 19 Documentation and React Server Components Specification," Meta Platforms Inc., 2026. [Trực tuyến]. Địa chỉ: https://react.dev. [Truy cập: 18-09-2026].

[3] The Movie Database, "TMDB API Reference Documentation v3," 2026. [Trực tuyến]. Địa chỉ: https://developer.themoviedb.org/docs. [Truy cập: 20-09-2026].

[4] Google Chrome Developers, "Web Vitals: Essential metrics for a healthy site," Google LLC, 2026. [Trực tuyến]. Địa chỉ: https://web.dev/vitals/. [Truy cập: 25-09-2026].

[5] Tailwind Labs, "Tailwind CSS v4 Documentation and Oxide Engine Architecture," 2026. [Trực tuyến]. Địa chỉ: https://tailwindcss.com/docs. [Truy cập: 22-09-2026].

[6] Microsoft Corp., "TypeScript Handbook: Strict Type Checking and Modern ECMAScript Modules," 2026. [Trực tuyến]. Địa chỉ: https://www.typescriptlang.org/docs/. [Truy cập: 21-09-2026].

[7] D. Abramov, L. Tan, S. Markbåge, and J. Savona, "Introducing Zero-Bundle-Size React Server Components," React, 2020. [Trực tuyến]. Địa chỉ: https://react.dev/blog/2020/12/21/data-fetching-with-react-server-components. [Truy cập: 24-09-2026].

[8] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, 2018. [Trực tuyến]. Địa chỉ: https://www.w3.org/TR/WCAG21/. [Truy cập: 26-09-2026].

[9] Node.js Foundation, "Node.js v22 LTS Domain Name System (DNS) Module Documentation," OpenJS Foundation, 2026. [Trực tuyến]. Địa chỉ: https://nodejs.org/api/dns.html. [Truy cập: 28-09-2026].

[10] IEEE Computer Society, "IEEE Standard for Software User Documentation," *IEEE Std 1063-2001 (R2007)*, pp. 1–45, 2007.

PHỤ LỤC

Phụ lục 1. Ranh giới Server/Client tại hàng phim

Ví dụ rút gọn từ `app/page.tsx`: Server Component tạo card; `MovieRow` phía client nhận phần tử đã tạo thay vì import file `MovieCard`. Cách truyền props/children này tuân theo hướng dẫn về interleaving của Next.js [1].

```
// app/page.tsx (Server Component)
<MovieRow
  title="Phim Thinh Hanh Trong Tuan"
  cards={trending.map((movie) => (
    <MovieCard key={movie.id} movie={movie} />
  ))}
/>
// components/client/MovieRow.tsx: "use client";
// Thanh cuon render phan tu da duoc truyen trong props.cards.
```

Phụ lục 2. Chọn Server Action hay Route Handler

Watchlist là thao tác ghi dữ liệu theo từng người dùng nên dùng `toggleWatchlist` trong `app/actions/watchlist.ts`; action kiểm tra `movieId`, ghi cookie và gọi `revalidatePath`. Live Search là thao tác đọc lặp lại khi nhập chữ nên dùng `GET /api/search?q=...`; client đợi 300ms và hủy request cũ bằng `AbortController`. Server kiểm tra từ khóa trước khi gọi TMDB. `s-maxage=120` là chỉ dẫn cho shared cache; mức độ cache thật cần kiểm tra trên môi trường triển khai.

Phụ lục 3. Cách tái lập các kiểm tra

Tại thư mục gốc của repo, chạy lần lượt:

```
npm ci
npm run lint
npx tsc --noEmit
npm test
npm run build
npm run start
```

Để xem route tĩnh/động, đọc bảng ở cuối output `npm run build`. Để kiểm tra Watchlist, mở một trang phim trên bản production, ghi trạng thái cookie và `/watchlist` trước/sau thao tác. Để kiểm tra streaming, dùng Network waterfall hoặc video khi làm chậm API diễn viên/phim tương tự; so sánh thời điểm fallback và nội dung. Để đo hiệu năng, dùng cùng

route, môi trường và cache state cho các lần đối chứng; giữ tệp kết quả thô. Không so sánh Lighthouse của trang chủ với FCP của trang chi tiết như cùng một phép đo.

Phụ lục 4. Khai báo sử dụng công cụ AI

Trong giai đoạn rà soát bản báo cáo và đối chiếu với mã nguồn, nhóm có sử dụng Codex để phát hiện mô tả không khớp code, gợi ý kiểm thử và hỗ trợ hiệu đính LaTeX. Các kết luận kỹ thuật trong bản nộp được đối chiếu lại với mã nguồn và tài liệu chính thức; các lệnh lint, type-check, test và build được chạy để xác minh. Công cụ AI không được xem là bằng chứng thay cho log, kết quả đo hoặc khả năng giải thích của từng thành viên. Mọi tài liệu và mã do bên thứ ba cung cấp vẫn phải được trích dẫn theo quy định của học phần.